funnelling

Marco Servetto
School of Engineering and Computer Science, Victoria University of Wellington, New Zealand
Colin S. Gordon
School of Computer & Information Sciences, Drexel University, United States of America

Abstract

...

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Funding *Colin S. Gordon*: US NSF CISE Award CCF-2007582

Acknowledgements I want to thank ...

1 Introduction

Java includes a useful feature called *method-local inner classes*, which has surprisingly subtle consequences. Like anonymous inner classes, this is a way for a method in a Java program to declare a class whose details are defined locally, entirely within the method, when the identity of the class is not needed outside the method's scope. However, these local classes are named, which allows them to be used with other language features exploiting nominal types. For example, one can cast objects to the method-local class type.

Unfortunately, as Servetto et al. [4] pointed out, there is an unsound interaction with generics. A non-generic method-local inner class `MyFoo` written in a generic method, or in a method of a generic class, can implicitly see the generic types in scope. Instances of `MyFoo` can then escape the lifetime of their enclosing method, where the generics had a certain runtime instantiation, and re-enter a second call of the same enclosing method, where the generics could have a different runtime instantiation. Servetto et al. probed the behaviour of JVM languages that support the same feature (Scala and Kotlin) and found that these had the same unsoundness problem for some use cases, but did flag concerns in other cases. However, this was not systematically investigated.

In this paper we clearly articulate the problem they observed, characterize the key properties that lead to the issue, and propose a static solution for future languages.

Example 1 demonstrates the issue.

```
// Example 1
public class Broken {
  static <X> Foo<X> m(Object other, final X x){
    class MyFoo extends Foo<X>{ public X x(){ return x; } }
    if(!(other instanceof MyFoo)){ return new MyFoo(); }
    return ((MyFoo)other); // cast passes with no warnings
  }
  public static void main(String[] a) {
    Foo<Integer> r1= Broken.<Integer>m("anything",5);
    Foo<String> r2= Broken.<String>m(r1,"Hi");
    System.out.println(r1.x()); //5
    System.out.println(r2.x()); //Throws ClassCastException: ..
  }
}
class Foo<X>{ public X x() {return x();} }
```

Line 4 declares a method-local inner class `MyFoo`, which depends on the generic parameter instantiation for `X` when the method is invoked. The `main` method invokes `m` twice. Line 9 invokes it for `Integer`s. Since the `other` argument for that call is not a `MyFoo`, the first invocation returns a `MyFoo` which is a `Foo<Integer>`. This result is then passed back into the second call to `m` on Line 10. This time, the `instanceof` check succeeds, because `other` in this case *is* a `MyFoo`, so the method (successfully) casts the `other` argument to `MyFoo`. Since `MyFoo` is declared as extending `Foo<X>`, this is a valid result for the function. Thus the `Foo<Integer>` that was passed as the first argument of the second call, on Line 10, is returned as a `Foo<String>`.

Clearly, this is wrong, because there are now two aliased variables of incompatible types; consequently, the later code fails. The failure itself is contrived. The two `println` calls actually dispatch to different overloads: the first to a version accepting an `Object`, and the second to the more specific overload expecting a `String`. The second call passes a value from a generic API to the specific overload, and this causes the Java compiler to insert the failing `checkcast` in the generated bytecode.

explain why (raw types)

A different version of Java could stop this problem at four points, two at compile time and two at run time:

- The `instanceof` check could have been statically flagged.
- The cast could have been statically flagged.
- The `instanceof` could have returned false in the second invocation, since it was used on a `MyFoo` from a different generic parameter instantiation.
- The cast could have failed, again because it was used on a `MyFoo` from a different generic parameter instantiation.

The source of this unsoundness is that the Java compiler does not treat `MyFoo` as generic. The generic type is seen as 'in scope', and it is 'captured' exactly as the values of local variables are captured. Within a single execution of the enclosing method, this is safe, because the type is constant. A `MyFoo` instance is conceptually an instance of a class that captures the generic type that is in scope when the instance is created. But when such a `MyFoo` instance escapes one execution of the enclosing method and enters a second execution of the same method, the instance is still logically connected to the same class type, with the original captured generic type. However, both the static type system and the run-time checks of casts and `instanceof` treat it as if it were connected to the new class type, with the new captured generic type.

Generalizing closure-capture behaviour to generic types as described above feels natural, but it is unsound in many situations. Closure capture is intrinsically private: a closure does not show in its external type signature what it has captured. Here it ends up being so private that the type system and the runtime have no access to it either. The static type system has to work on the publicly visible type, and in Java that type is not generic. This makes the compiler assume that the simplest nominal type identity is sufficient for casts and `instanceof` checks.

This problem touches on Java's erasure of generics, but only indirectly: even languages with reified generics store reified generics for the generic arguments exposed to the type system: a type like `List<Foo>` would get one reified generic, `Foo`, together with some language-mediated API to access it. A type like `MyFoo` would get no reified generics, since the generic type would have been captured and not exposed by the type. Thus, we argue that erasure versus reification is only marginally connected to this problem. The conceptual hurdle is in

the distinction between propagating the scope of generic types explicitly or implicitly.[1]

It is tempting to accept this as a quirk of Java, which is already known to have other sources of unsoundness [1]. However, this problem is broader in both scope and impact, as we show in the rest of the paper. This issue affects the statically typed JVM languages that support local classes, including Scala and Kotlin. It also propagates to compilers of those languages that do not target the JVM, such as Scala Native , and to ahead-of-time compilation of JVM bytecode, such as GraalVM's Native Image.

cite Scala Native

In the remainder of this paper:

- We detail the breadth of the problem, and how it is handled by various existing language implementations.
- We develop a core language modelling Java's generics, method-local classes and type tests, and show that typing method-local classes as Java does is unsound.
- We develop a new type system mechanism for systematically tracking dependence of a method-local class on generic parameters, and show that it soundly prevents the problem. Our solution, *funnelling*, makes the dependency of a method-local class on the generic types in scope explicit in its type: in Example 1 above, `MyFoo` would funnel `X` and become `MyFoo<X>`.

2 More Failing Examples

Example 1 is one of a family of programs that Java accepts with no warning and that fail at run time. This section shows more programs of this family.

2.1 Example 2: No Generic Supertype

In Example 1, the local class `MyFoo` mentions `X` in its supertype `Foo<X>`. Example 2 exercises the same fault without needing a generic supertype. Here `MyFoo` extends `Object`, and `X` occurs only in the return type of the method `x()` of `MyFoo`. The method `m` returns `Object`, so no type outside `m` mentions `X`.

```
1  // Example 2
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(!(other instanceof MyFoo)){ return new MyFoo(); }
6      MyFoo local= (MyFoo)other; // cast passes with no warnings
7      System.out.println("old was "+f.apply(local.x())); //ClassCastException
8      return local;
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

[1] Since JDK 16, with `Object o`, `o instanceof Foo<Integer>` is a compile-time error (before JDK 16, every non-reifiable `instanceof` target was an error), while the cast `(Foo<Integer>)o` is an unchecked warning; `instanceof` never warns. Modern Java accepts `o instanceof C<Ts>` only when the static type of `o` determines `Ts`; in this way an erased and a reified runtime give the same answer, as long as the program has no heap pollution. Example 1 creates heap pollution without any warning: `r2` has static type `Foo<String>`, but it refers to a `Foo<Integer>`.

The first call of `m` instantiates `X` with `Integer`, and returns a `MyFoo` whose method `x()` returns `5`. The second call of `m` instantiates `X` with `String`. In the second call, `other instanceof MyFoo` is true, and the cast `(MyFoo)other` at Line 6 succeeds with no warning. Then `local.x()` has static type `String`, but returns the `Integer 5`, and `f.apply(local.x())` at Line 7 throws a `ClassCastException`.

consider moving this level of detail (the cast is in a generated synthetic method) to the appendix. Maybe just say "the cast is in generated code" and put the rest in an appendix? Or footnote?

No instruction of `m` performs the failing cast. `javac` compiles the body of the lambda `s->s` into a private static method `lambda$main$1(String)` of the class `Broken`. At run time, the first evaluation of the lambda `s->s` calls `LambdaMetafactory`, which generates a class implementing `Function`; we call this class the *lambda class*. The method `apply(Object)` of the lambda class casts its argument to `String`, and then calls `lambda$main$1` with the result of that cast. Thus `f.apply(local.x())` passes the `Integer 5` to the method `apply` of the lambda class, and the cast to `String` in that method throws the `ClassCastException`.

The stack trace of the `ClassCastException` hides this. `LambdaMetafactory` defines the lambda class as a hidden class, and the JVM omits the frames of the methods of hidden classes from stack traces, unless the JVM runs with the diagnostic option `-XX:+ShowHiddenFrames`. So the topmost frame of the stack trace is the frame of `m`, at Line 7. The bytecode of `m` at Line 7 does contain a `checkcast` instruction to `String`, on the result of `f.apply`, but that instruction is never reached. The failure is detected in the lambda class: in code that the programmer did not write, at a point that depends on how javac and the JVM implement lambdas.

2.2 Example 3: No Explicit Cast

Example 3 exercises the same fault without needing an explicit cast. Java accepts Example 3 with no warning. Here we rely on the `other instanceof MyFoo local` test to bind `local` in one step.

```
1  // Example 3
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(other instanceof MyFoo local){ // no cast, no warnings
6        System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      }
8      return new MyFoo();
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`.

2.3 Example 4: Pattern Matching in `switch`

Colins ays: this should be one sentence after previous example that a switch can trigger the same thing, and ref to an appendix w/ code

Example 4 exercises the same fault with a `switch` instead of an `instanceof`. Here the case `case MyFoo local` performs the same type test as the pattern `other instanceof MyFoo local` of Example 3.

```
1  // Example 4
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

2.4 Example 5: No `checkcast` on the Result

this is redundant with example 2's lambda discussion. Maybe use this code for three examples: Object vs. generic type, switch instead of explicit instanceof, and lambda moving cast failure to synthesized method. actually the actual lambdas are different. Can we fold example 5 into example 2 by changing line 12 to ""+s.length"?

Example 5 exercises the same fault without a `checkcast` instruction on the result of `f.apply` in the bytecode of `m`. Example 5 could be expressed with either the `switch` of Example 4 or the `instanceof` of Example 3; here we chose the switch, but the other way would make the same point. Example 5 uses `Function<X,Integer>` and the lambda `s->s.length()` instead of `Function<X,String>` and the lambda `s->s`.

```
1  // Example 5
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,Integer> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->i);
16     Broken.<String>m(r1,"Hi",s->s.length());
17   }
18 }
```

The failure is the failure of Example 2: the method `apply` of the lambda class casts its argument, the `Integer 5`, to `String` before calling `lambda$main$1(String)`. The difference is only in the bytecode of `m`. Here the result of `f.apply` has static type `Integer`, and javac passes that result to the string concatenation without casting it to `Integer`, since any object can be converted to a `String`. So the bytecode of `m` contains no `checkcast` instruction on the result of `f.apply`. In Examples 2 to 4, the bytecode of `m` contains a `checkcast` instruction to `String` on the result of `f.apply`, but that instruction is never reached.

javac inserts a `checkcast` instruction where a value of an erased generic type is used at a more specific type. In Example 1, such a `checkcast` instruction detects the failure: javac inserts it in `main`, before the call of the overload `println(String)`. So one could expect a program with no `checkcast` instruction on the result of `f.apply` to fail later, in some virtual call, or not to fail at all. Example 5 shows that this expectation is wrong. In Examples 2 to 5, the value with the wrong type is the argument of `f.apply`: `local.x()` has static type `String`, but returns the `Integer 5`. The method `apply` of the lambda class casts that argument to `String` before `m` receives any result from `f.apply`. Thus the failure is detected at the same point in Examples 2 to 5, whether or not the bytecode of `m` contains a `checkcast` instruction on the result of `f.apply`.

2.5 Example 6: `X` Only in Private Members

Example 6 exercises the same fault without `X` in the public signature of `MyFoo`. Here `MyFoo` extends `Object`, and its only public method `show` takes a `MyFoo` and returns a `String`. `X` occurs only in the type of the private field `v`, and in the body of `show`, through the captured `f`.

focus is not really about private members, but that MyFoo's public signature doesn't mention X. Could have line 6 just capture x rather than storing in a field. Could turn example 2 into a family where we replace one line of example 2 with this show() definition we get this new example where X doesn't appear in the public signature.

```
1  // Example 6
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{
5        private final X v= x;
6        public String show(MyFoo o){ return f.apply(o.v); }
7      }
8      MyFoo mine= new MyFoo();
9      if(other instanceof MyFoo local){ // no cast, no warnings
10       System.out.println("old was "+mine.show(local)); //ClassCastException
11     }
12     return mine;
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

The first call of `m` returns a `MyFoo` whose field `v` holds the `Integer 5`. In the second call of `m`, `mine` is a new `MyFoo` of the second call, and `local` is the `MyFoo` of the first call. At Line 10, `mine.show(local)` applies the `f` of the second call, the lambda `s->s`, to the field `v` of `local`, the `Integer 5`. As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`. No type that a client of `MyFoo` can see mentions `X`, and still the two

instances of `MyFoo` disagree on `X`.

2.6 Counterexample 7: Class Generics

Counterexample 7 moves Example 3 into a generic class: `X` is a generic parameter of the class `Box` instead of a generic parameter of the method `m`.

```
1  // Example 7
2  class Box<X> {
3    final X x;
4    Box(X x){ this.x= x; }
5    Object m(Object other, Function<X,String> f){
6      class MyFoo{ public X x(){ return x; } }
7      if(other instanceof MyFoo local){ // error: Object cannot be safely cast to MyFoo
8        System.out.println("old was "+f.apply(local.x()));
9      }
10     return new MyFoo();
11   }
12 }
13 public class Broken {
14   public static void main(String[] a){
15     Object r1= new Box<Integer>(5).m("anything",i->""+i);
16     new Box<String>("Hi").m(r1,s->s);
17   }
18 }
```

Example 7 marks the boundary of the family: with class generics, javac does flag the fault. javac rejects Example 7 at Line 7, with the error `Object cannot be safely cast to MyFoo`. The method `m` is an instance method of `Box<X>`, so `MyFoo` is an inner class of `Box<X>`, with an enclosing instance of type `Box<X>`. javac types `MyFoo` as `Box<X>.MyFoo`, as javac types a member class `MyFoo` of `Box`. So `MyFoo` is generic in `X`, and `other instanceof MyFoo` is the uncheckable type test `other instanceof Box<X>.MyFoo`.

The cast is flagged too. The following method `m` replaces the method `m` of Example 7. The type test against `String` replaces the type test against `MyFoo`, which javac rejects, and the cast of Example 2 follows it.

```
1  // Example 7 with a cast: the method m of Box
2    Object m(Object other, Function<X,String> f){
3      class MyFoo{ public X x(){ return x; } }
4      if(other instanceof String){ return new MyFoo(); }
5      MyFoo local= (MyFoo)other; // warning: [unchecked] unchecked cast
6      System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      return local;
8    }
```

javac accepts this program, but reports an unchecked warning for the cast at Line 5. At run time the cast succeeds, and the program fails as Example 2 fails. So with class generics the program fails only after an unchecked warning, as a program with the cast `(List<String>)o` may fail. In Examples 1 to 6 instead, javac reports nothing.

Thus for the generic parameters of the enclosing class, Java already exposes the captured generic types in the type of the local class, through the type of its enclosing instance. Java does not do the same for the generic parameters of the enclosing method: no type of the local class carries the instantiation of the generic parameters of a method.

3 Funnelling

A natural suspicion is that this problem is "simply" a consequence of generic erasure: a language that cannot distinguish the generic instantiations of a class at run time must have difficulties of this sort. There is some truth to this suspicion: javac can accept a cast that the JVM cannot check, because of erasure. However, javac does flag uncheckable type tests: for `o` of type `Object`, javac reports an unchecked warning for the cast `(List<String>)o`, and an error for `o instanceof List<String>`. javac reports nothing for Examples 1 to 6 because `MyFoo` is not generic: the type test of `other` against `MyFoo` has no generic argument to leave unchecked. So reification does not change the situation by itself.

this paragraph may duplicate counter-example 7 text, otherwise we should move it there

The implementations of method-local classes, and of the similar features of other languages, differ in the granularity of the identity of the local class. The identity of the local class matters in two places: in the type system, where it decides whether two types of the local class are the same type, and at run time, where it decides the result of casts and type tests. There are three granularities:

1. **Identity as Declaration Point**: The declaration of a local class is a single point in the source code. So the local class `MyFoo` is one class, and the identity of `MyFoo` is its declaration point, the same for every instantiation of `m`.
2. **Identity as Generic Instantiation**: Generic classes and generic methods are functions from generic arguments to concrete classes and methods: `static <X> Object m(..)` declares infinitely many methods `m<Integer>`, `m<String>`, and so on, one for each instantiation of `X`. In the same way, the generic class declaration `class Foo<X>` declares infinitely many classes `Foo<Integer>`, `Foo<String>`, and so on. So each method `m<T>` declares its own local class `MyFoo`, since the body of each `m<T>` is the body of a different method.
3. **Identity as Dynamic Evaluation**: A declaration creates a new class, even for the same instantiation of the generic parameters. Scala has this granularity for captured values instead of captured types. An instance of an inner class `Inner` captures its enclosing instance `$outer`, and its type is the path-dependent type `a.Inner`. For the pattern `case i: a.Inner`, the Scala compiler emits an "outer test" that compares the captured `$outer` with `a`.[2] This is sound, but it is stricter than needed to avoid the problem at hand.

cite Scala outer tests

For method generics, the Java type system assumes granularity (1), as the casts do, so javac reports nothing and the program fails at run time. If the type of a local class contained the enclosing method and its generic instantiation, as in `Box<T1>.m<T2>.MyFoo`, then javac would flag the problem for generic methods too, as it does for generic classes. A version of Java with reified generics could solve the mismatch in Example 7, between the granularity (1) of the casts and the granularity (2) of the types. But reification would not change Examples 1 to 6, unless the types of local classes take the form `A<T1>.m<T2>.MyFoo`.

Granularities (1), (2) and (3) are points in a space of design options, but other points lie between them. Funnelling, the approach we present in this paper, lies between (1) and (2): in Example 1, `MyFoo` funnels `X` and becomes `MyFoo<X>`. For `other` of type `Object`, the existing rules of Java then flag `other instanceof MyFoo<X>` with an error and `(MyFoo<X>)other` with an unchecked warning, because `Object` does not determine `X`. These are the two compile-time points of the introduction. With reification, the same type test would be accepted by the compiler, and it would return false in the second call of `m`.

[2] Even here the type system exposes what is captured: inside the class of `a`, `Inner` means `this.Inner`.

We now show funnelling on the core language of Section 4, a version of Featherweight Java [3] rich enough to express the problem. The core language has generics, type tests and local classes. A local class is written as an expression, like an anonymous class in Java, but with a name: `new MyFoo<Fs> extends Foo<Ts>(es){Ms}` declares the local class `MyFoo` and creates an instance of it. This brings method-local classes into an expression-based calculus, without modelling statements. The funnels `Fs` are the generic parameters of `MyFoo`: a funnel `X funnel` funnels the type variable `X` in scope. During reduction, `X funnel` becomes the ground type that instantiates `X`. The type test `if (e instanceof T x){ e1 } else { e2 }` binds `x` in `e1`; the core language has no casts. In the listings, we omit the empty `<>` of methods and calls, and we write `<X>` for `<X extends Object<>>`. This is Example 1 in the core language, without funnels:

```
class A<> extends Object<>{}
class B<> extends Object<>{ B<> onlyB(){ return new B<>(); } }
class Foo<X> extends Object<>{ X x(){ return this.x(); } }
class Broken<> extends Object<>{
  <X> Foo<X> m(Object<> other, X x){ return
    if (other instanceof MyFoo<> w){ w }
    else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
}
```

As in Java, the type of the local class is `MyFoo<>`, with no generic parameters. Section 4 presents the type rules and the small step reduction of the core language. The type rules of Section 4 encode funnelling. With those type rules, the example above is ill typed: the supertype `Foo<X>` of `MyFoo` mentions `X`, but `MyFoo` does not funnel `X`. Rule NEW-LOCAL-JAVA enables the unsound Java typing:

$$\text{NEW-LOCAL-JAVA}\quad \frac{\Delta \vdash C'\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>}(es) : _ \qquad \Delta;\Gamma \setminus \texttt{this}, C\texttt{<>}\ \texttt{this} \vdash Ms\ \text{Ok} \qquad \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\} : C\texttt{<>}}$$

Rule NEW-LOCAL-JAVA uses the syntax `new C<> extends C'<Ts>(es){Ms}`. This syntax is an intermediate form between method-local classes and anonymous classes of Java. A method-local class `class C<> extends C'<Ts>{ C(){ super(es); } Ms }` is declared first, and then instantiated with `new C<>()`. An anonymous class `new C'<Ts>(es){Ms}` is declared and instantiated by a single expression. The syntax `new C<> extends C'<Ts>(es){Ms}` forms a (named) local class. A local class creates an instance as an anonymous class does, thus a local class is easy to use in an expression-based language. A local class also declares its own name, thus `instanceof` can test for a local class.

Rule NEW-LOCAL-JAVA types the methods Ms of the local class C under all the bounds Δ and all the variables Γ in scope, except `this`, which has type C`<>`. The example above is well typed using the type rules of Section 4 and rule NEW-LOCAL-JAVA. The main expression is the first line of the reduction below, and it has type `B<>`. We now show the small step reduction using the abbreviations `eA`, `vA` and `vB`, and omitting the renaming of bound variables:

```
eA = new Broken<>().<A<>>m(new Foo<A<>>(), new A<>())
vA = new MyFoo<> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }
vB = new MyFoo<> extends Foo<B<>>(){ B<> x(){ return new B<>(); } }

   new Broken<>().<B<>>m(eA, new B<>()).x().onlyB()
→* new Broken<>().<B<>>m(vA, new B<>()).x().onlyB()
→* (if (vA instanceof MyFoo<> w){ w } else { vB }).x().onlyB()
→ vA.x().onlyB()
→ new A<>().onlyB()
```

The first call of `m` reduces `eA` to `vA` with rules SIMPLEADDSUPER, SIMPLEFOUND and IFELSE. The type of `vA` is `MyFoo<>`: the instantiation `A<>` of `X` shows only in the supertype `Foo<A<>>` and in the method `x`. In the second call of `m`, rule IFTHEN checks only that the class of `vA` is a subclass of `MyFoo`. Thus the second call returns `vA`, a `Foo<A<>>`, as a `Foo<B<>>`. The last expression calls `onlyB` on an `A<>`, and `A<>` does not have a method called `onlyB`. Rule SEARCHSUPER looks for `onlyB` up to `Object<>`, where no rule applies: the reduction is stuck.

With funnelling, `MyFoo` must funnel `X`, written `new MyFoo<X funnel>`, and instances of the local class are of type `MyFoo<X>`. Rule CAST-DOWN accepts a type test only when the static type determines `X`, as Java does. Thus `other instanceof MyFoo<X> w` is ill typed for `Object<> other`, and the method `m` is ill typed. This is the rule by which Java rejects `o instanceof List<String>` for `Object o`. If we change the signature of `m` as follows, the method `m` is well typed:

```
  <X> Foo<X> m(Foo<X> other, X x){ return
    if (other instanceof MyFoo<X> w){ w }
    else { new MyFoo<X funnel> extends Foo<X>(){ X x(){ return x; } } }; }
```

Now `eA` is a well typed expression of type `Foo<A<>>`, but the main expression is ill typed: the second call of `m` expects a `Foo<B<>>` as the first argument, but `eA` is a `Foo<A<>>`.

With reification, the funnelled method `m` with `Object<> other` would be well typed, but the reduction would change. In the second call of `m`, the type test of the `MyFoo<A<>>` created by the first call against `MyFoo<B<>>` is false. The second call returns a new `MyFoo<B<>>`, and the main expression reduces to `new B<>()`. Reification without funnelling does not help: if we allow the value `vA` to have type `MyFoo<>`, even a reified type test of `vA` against `MyFoo<>` is true.

4 A Core Language for Funnelling

We now present the core language of Section 3: its syntax, its small step reduction and its type rules. The type rules encode funnelling. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA gives the Java typing of local classes.

4.1 Syntax

Expressions	e	::=	$x \mid e.\texttt{<}Ts\texttt{>}m(es) \mid e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es) \mid e.f \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}(es)$
			$\mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\{Ms\}$
			$\mid \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}$
Types	T	::=	$C\texttt{<}Ts\texttt{>} \mid X$
Funnels	F	::=	$G \mid X\ \texttt{funnel}$
Ground Types	G	::=	$C\texttt{<}Gs\texttt{>}$
Type Env.	Γ	::=	$T_1\ x_1, \ldots, T_n\ x_n$
Bounds	Δ	::=	$X_1\ \texttt{extends}\ T_1, \ldots, X_n\ \texttt{extends}\ T_n$
Declaration	Dec	::=	$\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ Ss\ \}$
(Field) Slots	S	::=	$T\ f\texttt{;}$
Methods	M	::=	$\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e\texttt{;}\ \}$
Class Decl.	K	::=	$\mathsf{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}$

An expression is a variable x, a method call $e.\texttt{<}Ts\texttt{>}m(es)$, a super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$, a field access $e.f$, an object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$ for a top-level type, a local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}$ or a type test $\texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}$.[3] As in FJ, `this` is just a special case of a variable x. A method call $e.\texttt{<}Ts\texttt{>}m(es)$ calls the method m on the receiver e, with type arguments Ts and arguments es. A super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$ searches the method m starting from the class of the type T, instead of from the class of the receiver e. That is, a super call works as a flexible kind of Java `super` call, attempting to call the method m as if the receiver were of the supertype T. An object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$ creates an instance of the top-level class $C\texttt{<}Ts\texttt{>}$. The arguments es are the values of the fields of C, followed by the arguments of the object creation of the superclass of C. A local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}$ declares the local class C and creates an instance of C.

The funnels Fs serve a double role. From the perspective of the inside of the local class (the methods Ms), the funnels Fs declare the type variables visible in the local class. From the perspective of the `new` expression itself, the funnels Fs instantiate those type variables, forming the actual type of the created instance.

To allow a local class instance to exist in the main expression, a funnel is either a ground type G or a funnelled type variable $X\ \texttt{funnel}$. In Decls, all the funnels are of the form $X\ \texttt{funnel}$, by well-formedness requirement A11. In a well-typed main expression, outside of the methods of local classes, all the funnels are of the form G: under the bounds $\emptyset$, the pairing of rule NEW-LOCAL is undefined for a funnel $X\ \texttt{funnel}$. When a local class is nested in a method of a local class in the main expression, a mixture of funnels $X\ \texttt{funnel}$ and G can be present.

A local class has no fields: the arguments es are the arguments of the object creation of the superclass $C'\texttt{<}Ts\texttt{>}$. The methods Ms of a local class can use the variables and the type variables in scope. The type test $\texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}$ reduces to e_1, with x bound to the value of e_0, when the class of the value of e_0 is a subclass of the class of T; otherwise the type test reduces to e_2.

A type is a class type $C\texttt{<}Ts\texttt{>}$ or a type variable X. A ground type G is a type with no type variables. A ground type is the equivalent of a value, but at the type level. The conventional type environment Γ gives a type to each variable in scope. Bounds Δ give an upper bound to

[3] The syntax `if (a instanceof B c)` is valid Java since Java 16.

each type variable in scope. A declaration `class` C`<`Δ`>` `extends` C'`<`Ts`>` `{` Ms Ss `}` declares the top-level class C with the type variables bounded by Δ, the superclass C'`<`Ts`>`, the methods Ms and the fields/slots Ss. A method `<`Δ`>`T_0 m`(`T_1 x_1`,` $\ldots$`,` T_n x_n`)` `{` `return` e`;` `}` has the type variables bounded by Δ, the return type T_0, the parameters $x_1, \ldots, x_n$ and the body e. Every method has a body; for simplicity, as in FJ, there are no abstract methods. A class declaration K is either a top-level declaration Dec or a local class.

A program is a class table Decls and a main expression e. We require the following well-formedness conditions:

(A1) In Decls, class names are unique across top-level and local classes.
(A2) In a class, all the field names, including the field names of the superclasses, are disjoint.
(A3) In a class, top-level or local, all the method names are disjoint.
(A4) In a method, all the parameter names $x_1, \ldots, x_n$ are disjoint.
(A5) In a Δ, all the X in dom(Δ) are disjoint.
(A6) In a list of funnels Fs, all the X in dom(Fs) are disjoint.
(A7) There is no shadowing of X or x, except for `this`.
(A8) No parameter and no variable of an `if` is `this`.
(A9) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.
(A10) In Decls, the superclass of every class, top-level or local, is a top-level class.
(A11) In Decls, every funnel is of the form X `funnel`.
(A12) The main expression has no free variable and no free type variable.

Well-formedness requirements A1-A10 and A12 are common requirements, such as uniqueness of names. A11 is the funnel requirement of Section 4.1. A method body of a local class also binds `this`, thus shadowing the `this` of the enclosing method.

4.2 Values and Contexts

$$
\begin{array}{rrcl}
\text{Values} & v & ::= & \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{)} \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}} \\
\text{Eval. Ctx.} & \mathcal{E} & ::= & \Box \mid \mathcal{E}\texttt{.<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{E}\ \texttt{instanceof}\ G\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
\text{Expr. Ctx.} & \mathcal{C} & ::= & \Box \mid \mathcal{C}\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{C}\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ \mathcal{C}\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ \mathcal{C}\ \texttt{\}}
\end{array}
$$

4.3 Declarations

Define K declared
Dec declared iff Dec $\in$ Decls

`new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` declared
where K declared
`<_>_` m`(_)` `{` `return` e`;` `}` $\in K$
`new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` occurs in e

362 **Define** $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts'$
$\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$
where `class` C`<`Δ`>` `extends _<_> { _ _ }` declared

363 $\mathsf{scope}(C_0\texttt{<}Ts\texttt{>}) = Fs \overset{\Delta,\Delta'}{\Longleftrightarrow} Ts$
where `<`Δ'`>_` m`(_) { return` e`; }` $\in C$
`new` C_0`<`Fs`>` `extends _<_>(_){_}` occurs in e
$\mathsf{scope}(\mathrm{generic}(C)) = \Delta; _$

364 **Define** $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts'$
$\emptyset \overset{\Delta}{\Longleftrightarrow} \emptyset = \emptyset; \emptyset$

365 X `funnel`$, Fs \overset{\Delta}{\Longleftrightarrow} T, Ts = X$ `extends` $\Delta(X), \Delta'; T, Ts'$
where $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts'$
$G, Fs \overset{\Delta}{\Longleftrightarrow} G, Ts = Fs \overset{\Delta}{\Longleftrightarrow} Ts$

366 **Define** $M \in K \qquad M \in C \qquad \mathrm{generic}(C) = T \qquad |F| = T$
$M \in K$ iff $M \in Ms$
where $K \in \{$`class _<_> extends _<_> {` Ms `_ }`$,$ `new _<_> extends _<_>(_){`Ms`}}`
$M \in C$ iff $M \in K$
where K declared
$K \in \{$`class` C`<_> extends _<_> { _ _ }`$,$ `new` C`<_> extends _<_>(_){_}}`

367 $\mathrm{generic}(C) = C$`<`$\mathrm{dom}(\Delta)$`>`
where `class` C`<`Δ`>` `extends _<_> { _ _ }` declared
$\mathrm{generic}(C) = C$`<`$\mathrm{dom}(Fs)$`>`
where `new` C`<`Fs`>` `extends _<_>(_){_}` declared
$|X$ `funnel`$| = X$
$|G| = G$

368 By well-formedness requirement A1, a class name C identifies at most one K with
369 K declared. $M \in K$ holds when M is a method of the class declaration K, top-level or
370 local. $M \in C$ holds when M is a method of the declared class C, top-level or local. For a
371 top-level class C with bounds Δ, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$. For a local class C with funnels
372 Fs, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0$, where Δ_0 is the subset of the bounds in scope in the body of
373 the method m declaring C, restricted to $\mathrm{dom}(Fs)$ and in the order of $\mathrm{dom}(Fs)$; note that
374 m could be a method of either a top-level or local class. Ts_0 is the subsequence of Ts, of
375 the same size as $\mathrm{dom}(Fs)$, holding the type argument of each funnelled X. For a top-level
376 class, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$ can return Δ and Ts of different lengths. The usage points of
377 scope use the notation $T[\Delta = Ts]$, the simultaneous substitution, defined only when Δ and
378 Ts have the same length; in the same way, $T[Fs = Ts]$ is defined only when Fs and Ts have
379 the same length. A simultaneous substitution, of types or of values, is defined only when the
380 names in its domain are distinct. A substitution $T[\Delta, \Delta' = Ts, Ts']$ is defined only when Δ
381 and Ts have the same length, and Δ' and Ts' have the same length.

382 For a type variable X, $\Delta(X)$ is the bound of X in Δ. $\mathrm{dom}(\Delta)$ is the sequence of the type vari-
383 ables in Δ, and $\mathrm{cod}(\Delta)$ is the sequence $T_1, \ldots, T_n$ of $\Delta = X_1$ `extends` $T_1, \ldots, X_n$ `extends` T_n.
384 For a variable x, $\Gamma(x)$ is the type of x in Γ. $\mathrm{dom}(Fs)$ is the sequence of the X such that
385 X `funnel` is in Fs. $\mathsf{FTV}(T)$ is the set of the type variables occurring in T. The figures leave
386 the following conventions implicit. A sequence $e_1, \ldots, e_n$, or es, separates its elements with
387 commas; a comma also concatenates two sequences, and either sequence may be empty. A
388 function or a judgement applied to sequences stands for one application at every position, and
389 requires the sequences to have the same length: $|F_1, \ldots, F_n|$ is $|F_1|, \ldots, |F_n|$; $\mathsf{FTV}(T_1, \ldots, T_n)$
390 is $\mathsf{FTV}(T_1) \cup \ldots \cup \mathsf{FTV}(T_n)$; $\Delta \vdash T_1, \ldots, T_n$ Ok is $\Delta \vdash T_1$ Ok $\quad \ldots \quad \Delta \vdash T_n$ Ok; $\Delta \vdash$
391 $T_1, \ldots, T_n <: T'_1, \ldots, T'_n$ is $\Delta \vdash T_1 <: T'_1 \quad \ldots \quad \Delta \vdash T_n <: T'_n$; $\Delta; \Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n$

is $\Delta;\Gamma \vdash e_1 : T_1 \quad \ldots \quad \Delta;\Gamma \vdash e_n : T_n$. Finally, $m \notin Ms$ holds when no method in Ms has name m, and $\mathsf{dom}(Ms)$ is the set of the names of the methods in Ms. $\Gamma \setminus x$ is Γ without the variable x. For a set of type variables Xs, $\Gamma|_{Xs}$ keeps the variables $T\ x$ of Γ such that $\mathsf{FTV}(T) \subseteq Xs$.

The pairing $Fs \overset{\Delta}{\Longleftrightarrow} Ts$ goes through the funnels Fs and the arguments Ts together, one position at a time, where $\emptyset$ is the empty sequence. At the position of a funnel X `funnel`, it adds X `extends` $\Delta(X)$ to the bounds and the argument to the types. At the position of a ground funnel G, the argument must be G itself, and nothing is added. An expression e' occurs in e when $e = \mathcal{C}[e']$, for an expression context $\mathcal{C}$. Expression contexts $\mathcal{C}$ do not enter the methods of a local class; thus $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ has at most one value.

4.4 Operational Semantics

We now present the reduction $e \to e'$ of the main expression. The reduction is small step, and uses the values v and the evaluation contexts $\mathcal{E}$. $e \to^* e'$ holds when e reduces to e' in zero or more steps.

Define $\mathsf{typeOf}(v) = G$
$\mathsf{typeOf}(v) = C\texttt{<}Gs\texttt{>}$
where $v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\}$

$$\boxed{e \to e'}$$

simpleFieldFound
$$\frac{\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f_i \in f_1, \ldots, f_n}{\texttt{new}\ C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\,.\,f_i \to v_i}$$

simpleFieldSuper
$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f \notin f_1, \ldots, f_n \qquad C \neq \texttt{Object}}{\texttt{new}\ C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\,.\,f \to \texttt{new}\ C'\texttt{<}Ts[\Delta = Gs]\texttt{>}(vs)\,.\,f}$$

innerFieldSuper
$$\frac{}{\texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>}(vs)\texttt{\{}Ms\texttt{\}}\,.\,f \to \texttt{new}\ C'\texttt{<}Gs'\texttt{>}(vs)\,.\,f}$$

ifThen
$$\frac{\emptyset \vdash \mathsf{typeOf}(v) <: C'\texttt{<_>} \qquad \mathsf{fresh}(e_1, e_1')}{\texttt{if}\ (v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x)\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_1'[x = v]}$$

ifElse
$$\frac{\emptyset \nvdash \mathsf{typeOf}(v) <: C'\texttt{<_>}}{\texttt{if}\ (v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x)\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_2}$$

$$\textsc{simpleAddSuper}\ \frac{v = \texttt{new}\ C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs)}$$

$$\textsc{simpleFound}\ \frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \{\ Ms\ _\ \}\ \mathsf{declared} \\ M \in Ms \qquad \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \}) \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

$$\textsc{innerFound}\ \frac{\begin{array}{c}v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}} \\ M \in Ms \qquad \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \}) \\ e = e_0[\Delta' = Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

$$\textsc{innerSuper}\ \frac{v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>(_)\{}Ms\texttt{\}} \qquad m \notin Ms}{v_0\texttt{.<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n)}$$

$$\textsc{searchSuper}\ \frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ _\ \}\ \mathsf{declared} \qquad m \notin Ms \qquad C \neq \texttt{Object}}{v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C'\texttt{<}Ts[\Delta = Gs]\texttt{>]<}Gs'\texttt{>}m(vs)} \qquad \textsc{ctx}\ \frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

Most of the rules are standard. Rules simpleFieldFound, simpleFieldSuper and innerFieldSuper read a field. Rule simpleFieldFound reads a field declared by the class of the object. Rule simpleFieldSuper reads a field not declared by the class C of the object from the object of the superclass of C, made of the remaining values. A local class declares no fields, so rule innerFieldSuper reads every field from the object of the superclass.

Rules ifThen and ifElse test the class of the value v against the class C' of the type test, ignoring the type arguments Ts of $C'\texttt{<}Ts\texttt{>}$. Rule ifThen reduces to e_1 with x replaced by v; rule ifElse reduces to e_2.

Rules simpleAddSuper, simpleFound, innerFound, innerSuper and searchSuper call a method. Rule simpleAddSuper turns a call on an object of a top-level class into a super call starting from the class of the object. Rule simpleFound runs the method m found in the class of a super call, replacing the type variables of the class and of the method, `this` and the parameters. Rule innerFound runs the method m found in a value of a local class; the funnelled type variables are already replaced in that value, so only the type variables of the method are replaced. Rule innerSuper turns a call on a value of a local class without the method m into a super call starting from its superclass. Rule searchSuper moves a super call to the superclass when the class does not declare m. Rule ctx reduces inside an evaluation context.

Rules ifThen, simpleFound and innerFound also use the relation fresh. The names bound in an expression e are the variable of every `if` occurring in e, and the names bound in every method of a local class occurring in e. The names bound in a method M are the type variables of its signature, its parameters, and the names bound in its body. $\mathsf{fresh}(e, e')$ holds when e' is e with the names bound in e renamed: e and e' are equal up to the names of their binders. $\mathsf{fresh}(M, M')$ holds when M' is M renamed in the same way. The variable `this` is never renamed. Renaming is needed because a value of a local class carries the code of its methods. For example, consider a local class `L` whose method `m` has a parameter `x` and calls

m on a captured value y of class L:

```
Object<> k(L<> y){ return
  new L<> extends Object<>(){ Object<> m(Object<> x){ return y.m(x); } }; }
```

In a call of k with a value v of class L, the body of m becomes v.m(x), and the method m of v binds x again. Without fresh, the substitution would attempt to place a binder of a name under a binder of the same name, breaking well-formedness requirement A7. Rules IFTHEN, SIMPLEFOUND and INNERFOUND place values under the binders of the expression they substitute into. Requirement A7 on the result requires those binders to differ from the names bound in the values. A substitution can place a value at several positions, so several occurrences of a value can bind the same names; those occurrences are never nested, so they do not shadow each other.

4.5 Typing

$$\boxed{\Delta;\Gamma \vdash e : T}$$

SUBS
$$\frac{\Delta;\Gamma \vdash e : T_1 \qquad \Delta \vdash T_2 \text{ Ok} \qquad \Delta \vdash T_1 <: T_2}{\Delta;\Gamma \vdash e : T_2}$$

VAR
$$\frac{}{\Delta;\Gamma \vdash x : \Gamma(x)}$$

NEW-OBJECT
$$\frac{}{\Delta;\Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

CALL
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts_0\texttt{>} \\ \mathsf{scope}(C_0\texttt{<}Ts_0\texttt{>}) = \Delta_0; Ts_2 \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in C_0 \\ \forall i \in 0..n.\, T_i' = T_i[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

DECLFIELD
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e : C\texttt{<}Ts'\texttt{>} \\ \texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ \texttt{_<_>\ \{\ _}\ Ss\ \texttt{\}}\ \text{declared} \\ T\ f\texttt{;} \in Ss\end{array}}{\Delta;\Gamma \vdash e.f : T[\Delta' = Ts']}$$

CALL-LB-RUNTIME
$$\frac{\begin{array}{c}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C_0\texttt{<}Ts_0\texttt{>} \\ \texttt{class}\ C_0\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ \texttt{_<_>\ \{\ _\ _\ \}}\ \text{declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in C_0 \\ \forall i \in 0..n.\, T_i' = T_i[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>\ \{\ _\ _\ \}}\ \text{declared} \\ \Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok} \qquad \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \Delta;\Gamma \vdash e_0, \ldots, e_n : C\texttt{<}Ts\texttt{>}, T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.[}C\texttt{<}Ts\texttt{>]<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

NEW-TOP

$$\frac{\texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ T_1\ f_1\texttt{;}\ \ldots\ T_n\ f_n\texttt{;}\ \}\ \text{declared} \qquad \Delta \vdash C\texttt{<}Ts'\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash e_1,\ldots,e_n : T_1,\ldots,T_n[\Delta' = Ts'] \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts[\Delta' = Ts']\texttt{>(}es\texttt{)} : _}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Ts'\texttt{>(}e_1,\ldots,e_n,es\texttt{)} : C\texttt{<}Ts'\texttt{>}}$$

NEW-LOCAL

$$\frac{\begin{array}{c}\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}\ \text{consistent} \\ Fs \overset{\Delta}{\Longleftrightarrow} |Fs| = \Delta';_ \qquad \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \\ \Delta' \vdash C\texttt{<}|Fs|\texttt{>}\ \text{Ok} \qquad \Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \\ \Delta';\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}, C\texttt{<}|Fs|\texttt{>}\ \texttt{this} \vdash Ms\ \text{Ok} \\ \mathsf{overrideOk}(\Delta', C_0\texttt{<}Ts\texttt{>}, Ms)\end{array}}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<}|Fs|\texttt{>}}$$

IF-INSTANCEOF

$$\frac{\Delta;\Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T\ \text{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \qquad \Delta;\Gamma, T\ x \vdash e_1 : T' \qquad \Delta;\Gamma \vdash e_2 : T'}{\Delta;\Gamma \vdash \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} : T'}$$

$$\boxed{\Delta \vdash T \rightsquigarrow T'}$$

CAST-UP

$$\frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'}$$

CAST-VAR

$$\frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

CAST-DOWN

$$\frac{\Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>} \qquad \mathrm{generic}(C) = C\texttt{<}Xs\texttt{>} \qquad \emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>} \qquad Xs \subseteq \mathsf{FTV}(Ts')}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

Define K consistent
$\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$ consistent
where $\texttt{new}\ C\texttt{<}Fs'\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts'\texttt{>(_)\{}Ms'\texttt{\}}$ declared
$Ts = Ts'[Fs' = |Fs|]$
$\mathrm{dom}(Ms') = \mathrm{dom}(Ms)$
$\mathsf{overrideOk}(C\texttt{<}|Fs|\texttt{>}, Ms)$

The rule NEW-LOCAL is the heart of funnelling, including subtle management of type environments. The rule constructs restricted bounds and a restricted type environment, reflecting only the explicitly-funnelled type variables: Δ' includes only the explicitly-funnelled type variables, and $\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}$ keeps only the variables of Γ, other than `this`, whose types mention only type variables of the result type $C\texttt{<}|Fs|\texttt{>}$ (which must be well-kinded under Δ'). The bounds Δ' are computed by $Fs \overset{\Delta}{\Longleftrightarrow} |Fs|$, which is undefined when a funnelled type variable is not in Δ. However, the rule still uses the *full* ambient environments for checking certain subexpressions. This is because some $e \in es$ may initially contain uses of *non*-funnelled type variables (e.g., in the type arguments of a method call), but will reduce to a value which no longer depends on such type variables. This is why the premise $\Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _$ checks the effective supertype instance under $\Delta;\Gamma$, but the premise $\Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \text{Ok}$ ensures the resulting type of that expression satisfies the funnelling constraints by checking kinding under the more restrictive Δ'.

A local class appears in two kinds of places: (1) The (original) declaration of the local class is written in the body of a method of a top-level class, possibly nested inside other local classes. (2) Copies of the local class can be in the main expression: reduction copies the declaration from the class table into the main expression, possibly inside the methods of another copy, renaming its bound names by fresh, replacing type variables with types and variables with values.

The relation K consistent trivially holds for case (1). For case (2), the first condition of the definition finds the declaration of K. The supertype and the method signatures of K must be those of the declaration, with the funnelled type variables of Fs' replaced by $|Fs|$. K and the declaration have the same method names. The bodies of the methods of K are not required to be those of the declaration: the main expression can hold a local class whose method bodies differ from the declaration, and rule NEW-LOCAL types those method bodies.

Rules CALL, SUB-INHERIT-LOCAL, WF-CLASS and CAST-DOWN read the declaration, while the reduction runs the methods of the copy.

The relation $\Delta \vdash T \rightsquigarrow T'$ is the *checked cast*: a type test against T' on a value of static type T is decided by the class of the value alone. Rule IF-INSTANCEOF accepts the type test `if (`e_0 `instanceof` T x`) {` e_1 `} else {` e_2 `}` only when $\Delta \vdash T_0 \rightsquigarrow T$, where T_0 is the type of e_0. Rule CAST-UP accepts a target T' that is a supertype of the source T. Rule CAST-VAR replaces a source type variable X with the bound of X. Rule CAST-DOWN accepts a target $C\texttt{<}Ts\texttt{>}$ that is a subtype of the source $C'\texttt{<}Ts_0\texttt{>}$, when the source determines the type arguments Ts. $\text{generic}(C) = C\texttt{<}Xs\texttt{>}$ is the class C instantiated with its own type variables Xs, and $C'\texttt{<}Ts'\texttt{>}$ is the supertype of $C\texttt{<}Xs\texttt{>}$ with class C'. The premise $\emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>}$ of rule CAST-DOWN is correctly called with $\emptyset$ as the bounds: a subtyping derivation between class types uses only rules SUB-REFL, SUB-INHERIT and SUB-INHERIT-LOCAL, which do not read Δ. Every type variable of Xs occurs in Ts', thus the type arguments Ts_0 of the source fix the type arguments Ts of the target. Thus a value of type $C'\texttt{<}Ts_0\texttt{>}$ whose class is a subclass of C has type $C\texttt{<}Ts\texttt{>}$, and rule IFTHEN compares only class names.

4.6 Typing of Programs, Classes and Methods

$\boxed{\vdash \mathsf{Decls}\ e\ \mathrm{Ok}}$

PROG-OK

$$\frac{\forall \mathsf{Dec} \in \mathsf{Decls}.\ \mathsf{Dec}\ \mathrm{Ok} \qquad \emptyset;\emptyset \vdash e : _}{\vdash \mathsf{Decls}\ e\ \mathrm{Ok}}$$

$\boxed{\Delta;\Gamma \vdash M\ \mathrm{Ok}}$

METHOD-OK

$$\frac{\Delta,\Delta' \vdash \mathrm{cod}(\Delta')\ \mathrm{Ok} \qquad \Delta,\Delta' \vdash T_0,\ldots,T_n\ \mathrm{Ok} \qquad \Delta,\Delta';\Gamma,T_1\ x_1,\ldots,T_n\ x_n \vdash e : T_0}{\Delta;\Gamma \vdash \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1,\ldots,T_n\ x_n\texttt{) \{ return } e\texttt{; \}}\ \mathrm{Ok}}$$

$\boxed{\mathsf{Dec}\ \mathrm{Ok}}$

CLASS-OK

$$\frac{\Delta \vdash \mathrm{cod}(\Delta)\ \mathrm{Ok} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>},T_1,\ldots,T_n\ \mathrm{Ok} \qquad \Delta;C\texttt{<}\mathrm{dom}(\Delta)\texttt{> this} \vdash Ms\ \mathrm{Ok} \qquad \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\texttt{class } C\texttt{<}\Delta\texttt{> extends } C'\texttt{<}Ts\texttt{> \{ }Ms\ T_1\ f_1\texttt{; }\ldots T_n\ f_n\texttt{; \}}\ \mathrm{Ok}}$$

Define $\mathsf{overrideOk}(\Delta, T, Ms)$ $\mathsf{overrideOk}(T, M)$

$\mathsf{overrideOk}(\Delta, T, Ms)$ iff $\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, M)$ for every $M \in Ms$ and every $C\texttt{<}Ts\texttt{>}$ such that $\Delta \vdash T <: C\texttt{<}Ts\texttt{>}$

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<_>}\ _\ m(_)\ \{\ _\ \})$
where $\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ _\texttt{<_>}\ \{\ Ms\ _\ \}$ declared
$m \notin Ms$

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m(T_1'\ x_1, \ldots, T_n'\ x_n)\ \{\ _\ \})$
where $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ _\texttt{<_>}\ \{\ Ms\ _\ \}$ declared
$\texttt{<}\Delta''\texttt{>}T_0\ m(T_1\ x_1', \ldots, T_n\ x_n')\ \{\ _\ \} \in Ms$
$\mathrm{cod}(\Delta'')[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0, \ldots, T_n[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m(T_1'\ x_1, \ldots, T_n'\ x_n)\ \{\ _\ \})$
where $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ _\texttt{<_>}(_)\{Ms\}$ declared
$\texttt{<}\Delta''\texttt{>}T_0\ m(T_1\ x_1', \ldots, T_n\ x_n')\ \{\ _\ \} \in Ms$
$\mathrm{cod}(\Delta'')[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0, \ldots, T_n[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

Rule CLASS-OK types the methods of a top-level class with `this` of type $C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}$. A method that a class declares again must keep the signature it has in every superclass, up to the names of its own type variables: CALL reads the signature from the class chosen by subsumption, while the reduction runs the method of the class of the receiver. The last case of $\mathsf{overrideOk}(T, M)$ compares a method of a local class with the method of the same name in the declaration of the local class, in the same way.

4.7 Kinding and Subtyping

$$\boxed{\Delta \vdash T\ \mathrm{Ok}}$$

WF-VAR
$$\frac{X\ \texttt{extends}\ _ \in \Delta}{\Delta \vdash X\ \mathrm{Ok}}$$

WF-CLASS
$$\frac{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0 \qquad \Delta \vdash Ts\ \mathrm{Ok} \qquad \Delta \vdash Ts_0 <: \mathrm{cod}(\Delta_0)[\Delta_0 = Ts_0]}{\Delta \vdash C\texttt{<}Ts\texttt{>}\ \mathrm{Ok}}$$

$$\boxed{\Delta \vdash T <: T'}$$

SUB-REFL
$$\frac{}{\Delta \vdash T <: T}$$

SUB-INHERIT
$$\frac{\texttt{class}\ C\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ _\ \}\ \text{declared} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}[\Delta_0 = Ts_0] <: T'}{\Delta \vdash C\texttt{<}Ts_0\texttt{>} <: T'}$$

SUB-INHERIT-LOCAL
$$\frac{\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(_)\{_\}\ \text{declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T'}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T'}$$

SUB-VAR
$$\frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

Rule WF-CLASS kinds both top-level and local classes, including a local class written in a method of another local class. Every type in Ts_0 is checked against the bound of its type variable in Δ_0. A funnelled type variable that is not in scope where its local class C is written leaves $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ undefined; thus no type $C\texttt{<}Ts\texttt{>}$ is well kinded.

4.8 Substitution of Type Variables

Define $e[Xs = Ts]$ $M[Xs = Ts]$ $F[Xs = Ts]$ $\Delta[Xs = Ts]$ $T[Xs = Ts]$

$$
\begin{array}{lcl}
x[Xs = Ts] & = & x \\
e.\texttt{<}Ts'\texttt{>}m\texttt{(}es'\texttt{)}[Xs = Ts] & = & e[Xs = Ts].\texttt{<}Ts'[Xs = Ts]\texttt{>}m\texttt{(}es'[Xs = Ts]\texttt{)} \\
e.\texttt{[}T'\texttt{]<}Ts''\texttt{>}m\texttt{(}es'\texttt{)}[Xs = Ts] & = & e[Xs = Ts].\texttt{[}T'[Xs = Ts]\texttt{]<}Ts''[Xs = Ts]\texttt{>} \\
 & & m\texttt{(}es'[Xs = Ts]\texttt{)} \\
e.f[Xs = Ts] & = & e[Xs = Ts].f \\
\texttt{new } C\texttt{<}Ts'\texttt{>(}es\texttt{)}[Xs = Ts] & = & \texttt{new } C\texttt{<}Ts'[Xs = Ts]\texttt{>(}es[Xs = Ts]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof }T'\ x\texttt{)} & = & \texttt{if (}e_0[Xs = Ts]\texttt{ instanceof }T'[Xs = Ts]\ x\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \} }[Xs = Ts] & & \texttt{\{ }e_1[Xs = Ts]\texttt{ \} else \{ }e_2[Xs = Ts]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts'\texttt{>} & = & \texttt{new } C\texttt{<}Fs[Xs = Ts]\texttt{> extends } C'\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [Xs = Ts] & & \texttt{(}es[Xs = Ts]\texttt{)\{}Ms[Xs = Ts]\texttt{\}} \\
X'\texttt{ funnel}[Xs = Ts] & = & X'\texttt{ funnel} \quad \text{if } X' \notin Xs \\
X_i\texttt{ funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i \\
G[Xs = Ts] & = & G \\
\texttt{<}\Delta\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_k'\ x_k\texttt{)} & = & \texttt{<}\Delta[Xs = Ts]\texttt{>}T_0'[Xs = Ts]\ m\texttt{(} \\
\texttt{\{ return }e\texttt{; \}}\ [Xs = Ts] & & \quad T_1'[Xs = Ts]\ x_1, \ldots, T_k'[Xs = Ts]\ x_k\texttt{)} \\
 & & \texttt{\{ return }e[Xs = Ts]\texttt{; \}} \\
 & & \quad \text{if } \mathrm{dom}(\Delta) \cap Xs = \emptyset \\
(X'\texttt{ extends }T')[Xs = Ts] & = & X'\texttt{ extends }T'[Xs = Ts] \\
C\texttt{<}Ts'\texttt{>}[Xs = Ts] & = & C\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
X'[Xs = Ts] & = & X' \quad \text{if } X' \notin Xs \\
X_i[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i
\end{array}
$$

Substitution replaces every occurrence of a type variable, *including* the ones in the method signatures of a local class. Substitution does not rename the type variables it meets. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground. Substitution of type variables is defined only when the type variables of the methods it enters are not in its domain. In the domain of a substitution, bounds Δ stand for $\mathrm{dom}(\Delta)$, and funnels Fs stand for their funnelled type variables: $T[F_1, \ldots, F_n = T_1, \ldots, T_n]$ replaces X_i with T_i for every $i \in 1..n$ such that $F_i = X_i\texttt{ funnel}$; the ground funnels are ignored. Thus $X_i\texttt{ funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] = T_i$ is a ground type G, and the result is again a funnel.

4.9 Substitution of Variables

Define $e[xs = vs]$ $M[xs = vs]$

$$
\begin{array}{lcl}
x[xs = vs] & = & x \quad \text{if } x \notin xs \\
x_i[x_1, \ldots, x_n = v_1, \ldots, v_n] & = & v_i \\
e.\texttt{<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{<}Ts\texttt{>}m(es'[xs = vs]) \\
e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{[}T\texttt{]<}Ts\texttt{>}m(es'[xs = vs]) \\
e.f[xs = vs] & = & e[xs = vs].f \\
\texttt{new } C\texttt{<}Ts\texttt{>(}es\texttt{)}[xs = vs] & = & \texttt{new } C\texttt{<}Ts\texttt{>(}es[xs = vs]\texttt{)} \\
\texttt{if (}e_0 \texttt{ instanceof } T\ x'\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [xs = vs] & = & \texttt{if (}e_0[xs = vs]\texttt{ instanceof } T\ x'\texttt{)} \\
 & & \texttt{\{ }e_1[xs = vs]\texttt{ \} else \{ }e_2[xs = vs]\texttt{ \}} \\
 & & \quad \text{if } x' \notin xs \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs = vs] & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
 & & \texttt{(}es[xs = vs]\texttt{)\{}Ms[xs' = vs']\texttt{\}} \\
 & & \quad \text{where } [xs' = vs'] \text{ is } [xs = vs] \text{ without the pair of } \texttt{this} \\
\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x'_1, \ldots, T_k\ x'_k) \\
\texttt{\{ return }e\texttt{; \}}\ [xs = vs] & = & \texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x'_1, \ldots, T_k\ x'_k) \\
 & & \texttt{\{ return }e[xs = vs]\texttt{; \}} \\
 & & \quad \text{if } x'_1, \ldots, x'_k \notin xs
\end{array}
$$

A local class binds `this`, as in Java: inside the methods of a local class, `this` is the local object, and the outer `this` is hidden. Thus substitution of variables enters the methods of a local class for every variable except `this`. Substitution of variables is defined only when the variable x' of an `if` and the parameters $x'_1, \ldots, x'_k$ of a method are not in xs.

...

4.10 Java Typing of Local Classes

Rule NEW-LOCAL-JAVA types local classes as Java does. A local class has no funnels, and its methods are typed under the full Δ and Γ in scope, except for `this`. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA makes our system unsound for the same reason Java is unsound. Consider the method `mk` below, the method `m` of Example 1 in our syntax:

```
<X> Foo<X> mk(Object<> other, X x){ return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
```

Rule NEW-LOCAL-JAVA gives to the object created by `mk` the type `MyFoo<>`. Rule SUB-INHERIT-LOCAL reads the declaration of `MyFoo`, so `MyFoo<>` is a subtype of `Foo<X>`, where `X` is the type variable of `mk`. The type `MyFoo<>` does not say which type `X` stood for when the object was created. Since generic(`MyFoo`) = `MyFoo<>` has no type variables, rule CAST-DOWN accepts the type test `other instanceof MyFoo<>` from `Object<>`. Thus the call `a.<String>mk(a.<Integer>mk(o, i), s)` has type `Foo<String>`, but it returns a `Foo<Integer>`. This is the unsoundness of Java. It is there even if we keep the restriction of Java that the name of a local class can only be written inside the method declaring it, as `mk` does. Our formal system shows two more failure points. They are the same failure, shown more explicitly:

- Reduction does not preserve types. The call `a.<Integer>mk(o, i)` has type `Foo<Integer>`. The value `o` is not a `MyFoo`, so the type test fails and the call takes the `else` branch. It reduces to the value

```
new MyFoo<> extends Foo<Integer>(){ Integer x(){ return i; } }
```

The type of this value is `MyFoo<>`, and `MyFoo<>` is not a subtype of `Foo<Integer>`. The supertypes of `MyFoo<>` come from the declaration of `MyFoo` in `mk`, not from the value: rule SUB-INHERIT-LOCAL derives $\emptyset \vdash$ `MyFoo<>` $<:$ `Foo<X>`, where `X` is the type variable of `mk`. Subtyping does not check kinding, so this judgement holds in the empty bounds $\emptyset$, where `X` is not in scope. However, `Foo<X>` is not well kinded under $\emptyset$, so rule SUBS cannot give the value the type `Foo<X>` either. Java has no typing of values during reduction, so in Java this failure is invisible.

Typing depends on the names of type variables. Our system lets the name of a local class be written anywhere. Consider the methods

```
<Y> MyFoo2<> mk2(Y y){
  return new MyFoo2<> extends Foo<Y>(){ Y x(){ return y; } }; }
<Y> Foo<Y> leak(MyFoo2<> z){ return z; }
```

The method `leak` is well typed, since the type variable `Y` of `leak` has the same name as the type variable `Y` of `mk2`. Renaming `Y` into `Z` in `leak` makes `leak` ill typed. The call

```
a.<String>leak(a.<Integer>mk2(i))
```

has type `Foo<String>`, but it returns a `Foo<Integer>`, without type tests.

Funnelling solves the problem in all three forms. Under rule NEW-LOCAL, the object created by `mk` has type `MyFoo<X>`, written `new MyFoo<X funnel> extends Foo<X>(){..}`. Since generic(`MyFoo`) = `MyFoo<X>`, rule CAST-DOWN accepts the type test `instanceof MyFoo<X>` from `Foo<X>`, but not from `Object<>`. Thus the funnelled `mk` takes `Foo<X> other` instead of `Object<> other`, and tests `other instanceof MyFoo<X> w`. The call `a.<Integer>mk(o, i)`, with `o` of type `Foo<Integer>`, reduces to a value of type `MyFoo<Integer>`, a subtype of `Foo<Integer>`.

...

briefly note that Java allows extra, “fresh” generic parameters in local classes

call out that we’re allowed to declare return types that mention an inner class; inner classes are “public”, and our approach works even in this case, the more restrictive case of Java is naturally covered

articulate early: It’s not just about generic reification: just checking type parameters aren’t enough, as our MyFoo cases have no generic params. You need to *deeply* check reified generics, you need to check not just the base type but also check the choice(s) of parent types from the context into the runtime check.

5 (Un)Soundness of Method Local Classes

With our language primitives in place, we now consider type safety of two variants of our language. The two variants differ only in the typing of local classes:

1. ζ(Java) types local classes with rule NEW-LOCAL-JAVA, as Java does;
2. ζ(funnelled) types local classes with rule NEW-LOCAL.

Both variants use the same type test: rule IF-INSTANCEOF accepts only checked casts, and at run time rules IFTHEN and IFELSE compare class names only. As shown in Section 4.10, ζ(Java) is unsound. We now show that ζ(funnelled) is sound.

▶ **Theorem 1** (Type Soundness). *Under a global* Decls *such that* Dec Ok *for every* Dec $\in$ Decls*: if* $\emptyset;\emptyset \vdash e_0 : T$*, and* $e_0 \to^* e$*, then* $\emptyset;\emptyset \vdash e : T$ *and either* e *is of form* v*, or there is an* e' *such that* $e \to e'$*.*

The reduction is deterministic, up to the names chosen by fresh.

▶ **Theorem 2** (Determinism). *If* $e \to e_1$ *and* $e \to e_2$*, then* $\mathsf{fresh}(e_1, e_2)$.

6 Related Work

It is not uncommon to have "intentional unsoundness" in industrial languages, such as Java's covariant array subtyping, or TypeScript's covariant method subtyping. However these are intentionally designed into the language to improve ease of use, and in most cases (like Java's array handling) problematic uses are detected at runtime (e.g., Java checks upon storing into an array that the new entry value is a subtype of the array's original content type).

However, unintentional unsoundness in industrial type systems is rarer. The most notable example is Amin and Tate's discovery [1] that Java and Scala's handling of generic type constraints was rendered unsound by the fact that `null` is treated as inhabiting all reference types, making all type constraints (even nonsensical ones) satisfiable. Servetto et al. [4] first pointed out the issue publicly, though in the Q&A session after the talk, multiple Java designers were unsurprised — as Gilad Bracha put it, "Once I saw the title of your talk, I understood what the problem was."

Colin says: As I listened to the Q&A session again, James framed this as a question of whether two instances allocated in different contexts should be considered the same or not. I think that might be useful framing to bring up early here. Gilad also pointed out that (paraphrasing) the BETA people knew an inner class was a property of the instance, not its own class.

discuss C# casts with reified generics

Flix is a functional language for the JVM. The Flix compiler monomorphizes polymorphic functions, as Rust and MLton do. Flix has no subtyping. The Flix construct closest to a local class is the `new` expression, which creates an object of an unnamed class that extends a Java class or implements a Java interface . A Flix program can not name such a class, so a Flix program can not write a type test against it, as with the generated classes of C#. Thus the type test of Example 1 can not be written in Flix.

cite Flix FAQ, for monomorphization and no subtyping

cite Flix anonymous classes

The key issues at work in the interaction of method-local inner classes and generics arise from the interaction of (1) nominal dynamic downcasts, (2) design choices about what runtime information about generic parameters is both persisted *and checked* by casts, and (3) (not) tracking all dependencies on type parameters in the type system. Essentially, Java and Scala both treat `MyFoo` as a sort of existential type — that there exists some `X` such that `MyFoo` extends `Foo<X>`. However, treating `MyFoo` as a closed rather than generic type is equivalent to assuming that there is only one existential for all instances — associated with the declaration site — rather than potentially different existentials for different instances. This combined with the fact that the JVM is unable to dynamically check for different instantiations leads to the unsoundness. Our dynamic solution resolves this with a specific implementation of casts on reified generics.

Modern Java: under modern Java there is no separate dynamic solution; see the note on the dynamic approach in the introduction.

Our static solution effectively prohibits the de facto existential capture of the enclosing type parameter.

The decision to treat types like the `MyFoo` variants in our examples as closed existential types that depend on other choices of types is not without parallels in other languages. Haskell and Rust, for example, include associated types [2], where a typeclass has a type

cite dot

as a member. A yet closer analogue is Scala's path-dependent types , where types may be *members* of an individual object, rather than associated with an individual class. The code below adapts the troublesome example method using Scala's type members:

```
trait Foo { type X; def x() : X; }
def m[Y](other: Any, newx: Y) : Foo2 =
    class MyFoo extends Foo { type X = Y; override def x() : X = newx; }
    newx match {
        case f: MyFoo => f
        case _ => MyFoo()
    }
```

This code compiles without warnings — and is not an issue. In this case the dependency of `MyFoo` instances on the type parameter used to call `m` is made explicit with a type member. But Scala now treats thes

7 Conclusions

write the conclusions

8 Technical Clarifications

The well-formedness requirements A1-A12 of Section 4 are part of the grammar. A metavariable ranges only over the syntactic forms that satisfy them. For example, by requirement A4, the grammar of M does not generate a sequence of symbols with the shape of a method declaring two parameters named `x`, so no metavariable M stands for that sequence of symbols. Thus every program Decls e satisfies the well-formedness requirements.

The type rules and the reduction rules are metarules. An instance of a metarule replaces every metavariable of the metarule, consistently, with a syntactic form of the grammar of that metavariable. A judgement holds when it is the conclusion of a derivation built from instances of the metarules. In particular, an instance of a reduction rule $e \to e'$ exists only when the right side e' is an expression. The reduction is the reduction of the main expression, so both sides of a step satisfy A12; an evaluation context binds no name, so the premise of rule CTX satisfies A12 too. Thus a reduction step never produces a sequence of symbols that the grammar does not generate. When the substitutions of a reduction rule are undefined for a redex, or would produce such a sequence of symbols, that reduction rule has no instance for that redex. Type Soundness implies that, under a Decls such that Dec Ok for every Dec $\in$ Decls, for every expression e with $\emptyset;\emptyset \vdash e : T$ that is not a value, some reduction rule has an instance with left side e.

9 Draft Notes

Marco had this idea that casts may be avoided by using dynamic dispatch returning this. Can not work, since Meth local class is private, there is no way an external interface can mention it to provide the cast method. Of course, the method local is private to the body (this means also the method signature can not use it) This means no self:Foo method, nor self:T method of a generic class eventually instantiating T=Foo.

This means the core language DO NEED if(e instanceof T x) else

//nested funnelling: new A<X>() B<X> f()-> new B<X>() new A<X>() <K>B<X,K> f()-> new B<X,K>()

using 'extends' for declarations in top level classes and generic methods using 'funnel' for named inner classes new A<X funnel Object>() B<X> f()-> new B<X funnel Object>() new A<X funnel Object>() <K extends Object> B<X,K> f()-> new B<X funnel Object,K funnel Object>() or, if the bounds are not repeated new A<X funnel>() <K extends Object> B<X,K> f()-> new B<X funnel,K funnel>() Very funky syntax: we get new B<X funnel Object,K funnel Object>() but, during reduction, this can become we get new B<Person,K funnel Object>() where we have a mixture of concrete types and generic funnelling

Global class definition uses class C<X extends Bar>.. named method local class uses new C<X funnel>().. normal new: new C<Ts>()

```
e ::= x | e.m(es) | e.f | new C<Ts>(es) | new C<Fs>(){Ms} | if (e0 instanceof T x){ e1 }else{ e2
T ::= C<Ts> | X
F ::= C<CCs> | X funnel// Important, can not be T | X funnel, otherwise we could have an X not 'f
CC::= C<CCs> //types with zero free type variables
B::=  X extends T // Need T, not F, to be able to do things like (Java) class Bar<X,Y extends X>
Dec::= class C<Bs>{ Ms Ss} //assume standard constructor
S ::= T x; //field/slot
M ::= Sig; | Sig{ return e; }
Sig::= <Bs> T m(x1:T1,..xn:Tn)
G ::= x1:T1..xn:Tn
```

//we do not need Object to be able to write 'extends Object' //this is exactly as the distinction of checked and unchecked exceptions: formally it evaporates in syntacitc sugar where all is checked //and all methods 'throws Error, RTE' // Can assume the user provides their own Object type Note: an use of X is just X, a declaration of X is X extends T; a funnelling of X is X funnel Program is just many Dec plus an e typable in empty MARCO TODO: typechecking Decs;Bs;G|-e:T Decs|-Dec:ok Decs;T|-M:ok e –> e' Note, there are multiple semantics/ts: e instanceof CC //always clear e instanceof T with T not of form CC would not typecheck

e –> e' for Java with warn->err: just have CC in instanceof Java with magic reifined cast: just have T in instanceof (since reduction will replace(thus remember) generics Java with warn->off: AT RUN TIME instanceof only checks base class, ignores generic parameters AT COMPLILE TIME, instanceof can be err if the full type (including generics) is not potentially correlated)?? unobvious consider interfaces In prose, should explain that we won't flag cast of some type Foo to some type Bar, even though Java sometimes flags unrelated casts

Modern Java: these three semantics are replaced by the single one of Java 16+: erased run-time check, plus the checked-cast premise of rule IF-INSTANCEOF (Section 4.5). Unrelated types are rejected by it, as in Java.

<X extends List<Y funnel»
class Foo<X> class Getter<X funnel> <Y extends List<X» X getFirst(Y y) y.getOrElse(0,null)

<Y> .. return foo<Y,MyX>(); <Y,X extends List<Y» foo() return Foo<X funnel>()...
X ;

A Lean Formalization

This appendix relates the formalism of Section 4 to a Lean 4 formalization in the folder `lean` of the paper repository. Rule NEW-LOCAL-JAVA, shown in Section 3 and used in Section 4.10,

is not formalized: Type Soundness concerns only the type system with rule NEW-LOCAL. The Lean formalization is set up, but the proofs of the soundness theorems are not done yet.

A.1 Request

This appendix answers the following request, reported verbatim, so that we can check later how much of it is addressed.

```
The Appendix, for each piece of formalism we report the following:

(A)Original formalism

(B)Corresponding Lean formalism

(C)English explanation of the differences and either
- why it ok for those differences to be there.
- or what the more appropriate form of the original formalism should be

(D)A clear report of the lean statements and an high level description of the
    proofs.

(E)A list of validation examples. Those are:
- programs that Do pass the lean type system AND the paper type system, or that do
    not pass both the lean type system and the paper type system.
- programs that reduce in some steps equally in both the paper type system and the
    lean type system. Basically, this is 'testing': we are empirically checking
    that the two formalism are the same by checking that they behave the same on a
     range of examples including interesting corner cases.
```

A.2 Status

The file `lean/lean-toolchain` selects Lean 4.35.0: elan runs that version for every command in the folder `lean`. The project depends on Mathlib v4.35.0, which brings Batteries and Aesop; `lake exe cache get` downloads Mathlib already compiled. The current files import only the core library of Lean. `lake build` in the folder `lean` checks all the files.

The formalization has two levels. The *high level*, `lean/Funnelling/High`, follows the metarules of the paper as closely as Lean allows; Section A.4 compares it with the paper. The *low level*, `lean/Funnelling/Low`, is free: it can change in any way that makes the proofs easier, and several low levels can be tried, without changing the high level. The low level starts as a computable version of the high level. The files are:

- `lean/Funnelling/High/Syntax.lean`: names, types, ground types, funnels, expressions, methods, values, declarations, class declarations K.
- `lean/Funnelling/High/Substitution.lean`: substitution of type variables and of variables, and fresh.
- `lean/Funnelling/High/Declarations.lean`: expression contexts, declared, $M \in C$, generic, the pairing, scope, typeOf.
- `lean/Funnelling/High/WellFormed.lean`: the well-formedness requirements A1-A12.
- `lean/Funnelling/High/Typing.lean`: subtyping, kinding, checked casts, overrideOk, consistent, typing, method, class and program typing.
- `lean/Funnelling/High/Reduction.lean`: evaluation contexts and the reduction.

- `lean/Funnelling/Low/*.lean`: the low level, and `lean/Funnelling/Low/Soundness.lean`, progress and preservation of the low level.
- `lean/Funnelling/Translation.lean`: the translation from the high level to the low level.
- `lean/Funnelling/Adequacy.lean`: lemmas that relate the two levels through the translation.
- `lean/Funnelling/Proof.lean`: the proof of Type Soundness.
- `lean/Funnelling/Soundness.lean`: Type Soundness, on the high level.
- `lean/Funnelling/Examples.lean`: the validation examples, on the high level.

Lean accepts all the files. The fixed statements are `typeSoundness` and `determinism` in `lean/Funnelling/Soundness.lean`, the two theorems of the paper. Everything else is a component of their proofs, and can change: the low level, the translation, the adequacy lemmas and `lean/Funnelling/Proof.lean`. `lean/Funnelling/Proof.lean` proves `typeSoundness` from progress and preservation on the high level, each proved through the low level; the low-level progress and preservation and the six adequacy lemmas are stated with `sorry`. Section A.6 lists which validation examples are proved and which are stated with `sorry`.

The Lean code is written in ASCII: `->` is $\to$, `/\` is $\wedge$, `\/` is $\vee$, `List.Mem x xs` is $x \in xs$, and `Delta`, `gamma`, `sigma` are Δ, Γ, σ.

A.3 Differences Common to All the Pieces

The following differences hold for every piece below, and the pieces do not repeat them.

- **Metavariables.** Each metavariable of the paper is a Lean type: `ClassName` (C), `MethName` (m), `FieldName` (f), `VarName` (x), `TVarName` (X), `Ty` (T), `GTy` (G), `Funnel` (F), `Value` (v), `Expr` (e), `Method` (M), `Dec`, `K`. A ground type is also a type, and a value is also an expression: Lean inserts the conversions `GTy.ty` and `Value.expr`, and `GTy.tys` and `Value.exprs` for lists, where needed. A rule written with G or v in the paper is written with a `GTy` or a `Value` in Lean. A Lean constructor has the name of the rule of the paper: `subInherit` is SUB-INHERIT, `callLbRuntime` is CALL-LB-RUNTIME.
- **Grammar and well-formedness.** In the paper, the well-formedness requirements restrict the grammar (Section 8). Lean defines the syntax as inductive types, and A1-A12 as the propositions `WFDecls` and `WFExpr`. Type Soundness assumes them for the initial program, and Determinism for the expression that reduces. A step $e \to e'$ of the paper is `Reduces d e e'`: a `Step` whose result satisfies `WFExpr`, since an instance of a rule of the paper exists only when its result is an expression, and the result of a step is the new main expression (A12). The premise of `Step.ctx` is a `Step`: an evaluation context binds no name, so `WFExpr (E.plug e')` implies `WFExpr e'`.
- **The class table.** Decls is an explicit argument `d` of every judgement.
- **Sequences.** A judgement on sequences has its own name: $\Delta \vdash Ts$ Ok is `KindSeq d Delta Ts`, $\Delta \vdash Ts <: Ts'$ is `SubSeq d Delta Ts Ts'`, and $\Delta;\Gamma \vdash es : Ts$ is `TypingSeq d Delta gamma es Ts`; $\Delta;\Gamma \vdash Ms$ Ok is a premise for every method.
- **Substitution and lengths.** The paper defines $T[\Delta = Ts]$ only when Δ and Ts have the same length. $T[\Delta = Ts]$ is `T.subst (bind Delta Ts)` and $T[Fs = Ts]$ is `T.subst (bindF Fs Ts)`; the rule has the premise that the lengths are the same. The paper defines a simultaneous substitution only when the names in its domain are distinct. Rules SIMPLEFOUND and INNERFOUND have this as premises (`Nodup`), since their domains hold the names chosen by fresh; in the other rules the domain is distinct by A5, A6 and A7.

- **Definitions.** A **Define** box of the paper with *where* clauses (declared, $M \in C$, generic, scope, the pairing, overrideOk(T, M)) is an inductive relation, one constructor per clause. A **Define** box that defines a function (dom, cod, $|F|$, typeOf, the substitutions) is a function.

A.4 Side by Side: Paper and Lean

Each block below shows, on the left, a piece of the formalism of the paper, with the minimal text of the paper that the piece needs; on the right, the Lean definitions of the same concepts in the high level `Funnelling.High`; under them, the words that map one side to the other, and the differences beyond Section A.3.

A.4.1 Syntax

Names, types, ground types, funnels

$$\begin{array}{lcl} T & ::= & C\texttt{<}Ts\texttt{>} \mid X \\ F & ::= & G \mid X \texttt{ funnel} \\ G & ::= & C\texttt{<}Gs\texttt{>} \end{array}$$

A ground type G is a type with no type variables.

```
structure ClassName where
  name : String
-- MethName, FieldName, VarName, TVarName: the same
inductive Ty where
  | cls (C : ClassName) (Ts : List Ty)
  | var (X : TVarName)
inductive GTy where
  | cls (C : ClassName) (Gs : List GTy)
inductive Funnel where
  | ground (G : GTy)
  | funnel (X : TVarName)
instance : Coe GTy Ty where
  coe := GTy.ty
instance : Coe (List GTy) (List Ty) where
  coe := GTy.tys
```

Mapping. `GTy.ty` turns G into the same type T.

Bounds and type environments

$$\begin{array}{lcl} \Gamma & ::= & T_1\ x_1, \ldots, T_n\ x_n \\ \Delta & ::= & X_1 \texttt{ extends } T_1, \ldots, X_n \texttt{ extends } T_n \end{array}$$

For a type variable X, $\Delta(X)$ is the bound of X in Δ.

```
structure Bound where
  X : TVarName
  T : Ty
structure VarDecl where
  T : Ty
  x : VarName
abbrev Bounds := List Bound
abbrev Gamma := List VarDecl
def Gamma.lookup (gamma : Gamma) (x : VarName) : Option
    Ty :=
  (gamma.find? fun d => d.x == x).map VarDecl.T
```

Mapping. X `extends` T is `{ X := X, T := T }`; $T\ x$ is `{ T := T, x := x }`; $\Gamma(x)$ is `gamma.lookup x`.

Expressions and methods

$$
\begin{array}{rcl}
e & ::= & x \mid e.\texttt{<}Ts\texttt{>}m(es) \mid e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es) \\
 & & \mid e.f \mid \texttt{new}\ C\texttt{<}Fs\texttt{>} \\
 & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\{Ms\} \\
 & & \mid \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \\
M & ::= & \texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e;\ \}
\end{array}
$$

```
mutual
inductive Expr where
  | var (x : VarName)
  | call (e : Expr) (Ts : List Ty) (m : MethName) (es :
      List Expr)
  | superCall (e : Expr) (T : Ty) (Ts : List Ty) (m :
     MethName) (es : List Expr)
  | field (e : Expr) (f : FieldName)
  | new (C : ClassName) (Ts : List Ty) (es : List Expr)
  | newLocal (C : ClassName) (Fs : List Funnel) (C' :
     ClassName) (Ts : List Ty)
      (es : List Expr) (Ms : List Method)
  | ifInst (e0 : Expr) (T : Ty) (x : VarName) (e1 e2 :
     Expr)
inductive Method where
  | mk (bounds : Bounds) (T0 : Ty) (m : MethName) (
     params : List VarDecl) (body : Expr)
end
```

Mapping. The super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$ is `superCall`; the local class is `newLocal`; the type test is `ifInst`.

Values

$$
v ::= \texttt{new}\ C\texttt{<}Gs\texttt{>}(vs) \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>}(vs)\{Ms\}
$$

```
inductive Value where
  | new (C : ClassName) (Gs : List GTy) (vs : List
     Value)
  | newLocal (C : ClassName) (Gs : List GTy) (C' :
     ClassName) (Gs' : List GTy)
      (vs : List Value) (Ms : List Method)
instance : Coe Value Expr where
  coe := Value.expr
instance : Coe (List Value) (List Expr) where
  coe := Value.exprs
```

Mapping. `Value.expr` turns v into the same expression; the funnels Gs of a value of a local class become ground funnels.

Declarations and programs

$$
\begin{array}{rcl}
\mathsf{Dec} & ::= & \texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ Ss\ \} \\
S & ::= & T\ f; \\
K & ::= & \mathsf{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}
\end{array}
$$

A program is a class table Decls and a main expression e.

```
structure Slot where
  T : Ty
  f : FieldName
structure Dec where
  C : ClassName
  bounds : Bounds
  C' : ClassName
  Ts : List Ty
  Ms : List Method
  Ss : List Slot
inductive K where
  | dec (D : Dec)
  | loc (C : ClassName) (Fs : List Funnel) (C' :
     ClassName) (Ts : List Ty)
      (es : List Expr) (Ms : List Method)
abbrev Decls := List Dec
```

Mapping. Δ of a declaration is `bounds`.

Evaluation contexts

$$\begin{array}{rcl}\mathcal{E} & ::= & \square \mid \mathcal{E}.\texttt{<}Gs\texttt{>}m(es) \mid v.\texttt{<}Gs\texttt{>}m(vs, \mathcal{E}, es) \\ & & \mid \mathcal{E}.[G]\texttt{<}Gs\texttt{>}m(es) \\ & & \mid v.[G]\texttt{<}Gs\texttt{>}m(vs, \mathcal{E}, es) \\ & & \mid \mathcal{E}.f \mid \texttt{new } C\texttt{<}Gs\texttt{>}(vs, \mathcal{E}, es) \\ & & \mid \texttt{new } C\texttt{<}Gs\texttt{> extends } C'\texttt{<}Gs'\texttt{>}(vs, \mathcal{E}, es)\{Ms\} \\ & & \mid \texttt{if } (\mathcal{E} \texttt{ instanceof } G\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}\end{array}$$

```
inductive ECtx where
  | hole
  | callRecv (E : ECtx) (Gs : List GTy) (m : MethName)
     (es : List Expr)
  | callArg (v : Value) (Gs : List GTy) (m : MethName)
     (vs : List Value) (E : ECtx)
     (es : List Expr)
  | superRecv (E : ECtx) (G : GTy) (Gs : List GTy) (m :
     MethName) (es : List Expr)
  | superArg (v : Value) (G : GTy) (Gs : List GTy) (m :
     MethName) (vs : List Value)
     (E : ECtx) (es : List Expr)
  | field (E : ECtx) (f : FieldName)
  | newArg (C : ClassName) (Gs : List GTy) (vs : List
     Value) (E : ECtx) (es : List Expr)
  | localArg (C : ClassName) (Gs : List GTy) (C' :
     ClassName) (Gs' : List GTy)
     (vs : List Value) (E : ECtx) (es : List Expr) (Ms
        : List Method)
  | ifCond (E : ECtx) (G : GTy) (x : VarName) (e1 e2 :
     Expr)
```

Mapping. $\mathcal{E}[e]$ is `E.plug e`.

A.4.2 Well-Formedness

A1, A2, A3

(A1) In Decls, class names are unique across top-level and local classes.
(A2) In a class, all the field names, including the field names of the superclasses, are disjoint.
(A3) In a class, top-level or local, all the method names are disjoint.

```
abbrev A1 (d : Decls) : Prop := ((classes d).map K.name
   ).Nodup
abbrev A2 (d : Decls) : Prop :=
  forall D, List.Mem D d -> ((superChain d (d.length +
     1) D.C).flatMap fun D' => D'.Ss.map Slot.f).Nodup
abbrev A3 (ks : List K) : Prop := forall k, List.Mem k
   ks -> (k.Ms.map Method.name).Nodup
```

Mapping. `classes d` lists the top-level classes and every local class written in their methods. `superChain d n C` is the chain of superclasses of C, cut after n classes; an acyclic chain is never longer than |Decls|, and A9 rejects a cyclic one.

A4, A5, A6

(A4) In a method, all the parameter names $x_1, \ldots, x_n$ are disjoint.
(A5) In a Δ, all the X in dom(Δ) are disjoint.
(A6) In a list of funnels Fs, all the X in dom(Fs) are disjoint.

```
abbrev A4 (Ms : List Method) : Prop := forall M, List.
   Mem M Ms -> (M.params.map VarDecl.x).Nodup
abbrev A5 (Deltas : List Bounds) : Prop := forall Delta
   , List.Mem Delta Deltas -> (dom Delta).Nodup
abbrev A6 (ks : List K) : Prop := forall k, List.Mem k
   ks -> (domF k.Fs).Nodup
```

Mapping. Each requirement is applied to all the methods, bounds and local classes of Decls, and of the main expression.

A7

(A7) There is no shadowing of X or x, except for `this`.

A method body of a local class also binds `this`, thus shadowing the `this` of the enclosing method; this is the only form of shadowing.

```
def Expr.noShadow (Xs : List TVarName) (xs : List
   VarName) : Expr -> Bool
  ...
  | .newLocal _ _ _ _ es Ms =>
    Expr.noShadowList Xs xs es &&
    Method.noShadowList Xs (xs.filter fun x => x !=
       this) Ms
  | .ifInst e0 _ x e1 e2 =>
    e0.noShadow Xs xs && !(xs.contains x) && e1.
       noShadow Xs (x :: xs) &&
    e2.noShadow Xs xs
def Method.noShadow (Xs : List TVarName) (xs : List
   VarName) : Method -> Bool
  | .mk Delta _ _ ps e =>
    (dom Delta).all (fun X => !(Xs.contains X)) &&
    (ps.map VarDecl.x).all (fun x => !(xs.contains x))
       &&
    e.noShadow (Xs ++ dom Delta) (this :: xs ++ ps.map
       VarDecl.x)
abbrev A7Decls (d : Decls) : Prop :=
  forall D, List.Mem D d -> Method.noShadowList (dom D.
     bounds) [] D.Ms = true
```

Mapping. `Xs` and `xs` are the type variables and the variables in scope; the cases written `...` go into the subexpressions.

Funnels bind nothing: X `funnel` refers to the X in scope.

A8, A9, A10, A11

(A8) No parameter and no variable of an `if` is `this`.
(A9) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.
(A10) In Decls, the superclass of every class, top-level or local, is a top-level class.
(A11) In Decls, every funnel is of the form X `funnel`.

```
abbrev A8 (Ms : List Method) (ifVars : List VarName) :
   Prop :=
  (forall M, List.Mem M Ms -> Not (List.Mem this (M.
     params.map VarDecl.x))) /\ Not (List.Mem this
     ifVars)
abbrev A9 (d : Decls) : Prop :=
  (exists D, List.Mem D d /\ D.C = Object /\ D.bounds.
     isEmpty /\ D.C' = Object /\ D.Ts.isEmpty /\
    D.Ms.isEmpty /\ D.Ss.isEmpty) /\
  forall D, List.Mem D d -> exists D', List.Mem D' (
     superChain d (d.length + 1) D.C) /\ D'.C = Object
abbrev A10 (d : Decls) : Prop := forall k, List.Mem k (
   classes d) -> (findDec d k.sup).isSome
abbrev A11 (d : Decls) : Prop := forall k, List.Mem k (
   classes d) -> forall F, List.Mem F k.Fs -> F.
   isFunnel
```

Mapping. `ifVars` lists the variables of all the `if`.

A12

(A12) The main expression has no free variable and no free type variable.

```
def Expr.closed (Xs : List TVarName) (xs : List VarName
   ) : Expr -> Bool
  | .var x => xs.contains x
  | .newLocal _ Fs _ Ts es Ms =>
    (domF Fs ++ Ty.ftvList Ts).all Xs.contains && Expr.
       closedList Xs xs es && Method.closedList Xs xs
       Ms
  | .ifInst e0 T x e1 e2 =>
    e0.closed Xs xs && T.ftv.all Xs.contains && e1.
       closed Xs (x :: xs) && e2.closed Xs xs
  | ...
def Method.closed (Xs : List TVarName) (xs : List
   VarName) : Method -> Bool
  | .mk Delta T0 _ ps e =>
    (Ty.ftvList (T0 :: cod Delta ++ ps.map VarDecl.T)).
       all (Xs ++ dom Delta).contains &&
    e.closed (Xs ++ dom Delta) (this :: xs ++ ps.map
       VarDecl.x)
```

Mapping. `Xs` and `xs` are the type variables and the variables in scope; the cases written `...` check the types they hold and go into the subexpressions.

All the requirements

We require the well-formedness conditions A1-A12.

The well-formedness requirements A1-A12 are part of the grammar (Section 8).

```
abbrev WFDecls (d : Decls) : Prop :=
  A1 d /\ A2 d /\ A3 (classes d) /\ A4 d.methods /\
  A5 (d.map Dec.bounds ++ d.methods.map Method.bounds)
     /\ A6 (classes d) /\
  A7Decls d /\ A8 d.methods (d.flatMap fun D => Method.
     ifVarsList D.Ms) /\
  A9 d /\ A10 d /\ A11 d
abbrev WFExpr (e : Expr) : Prop :=
  A3 e.locals /\ A4 e.methods /\ A5 (e.methods.map
     Method.bounds) /\ A6 e.locals /\
  e.noShadow [] [] = true /\ A8 e.methods e.ifVars /\ e.
     closed [] [] = true
```

Mapping. `WFExpr` applies to the main expression the requirements that restrict it, A3-A8 and A12.

A.4.3 Declarations

Expression contexts and *occurs in*

$$
\begin{array}{rcl}
\mathcal{C} & ::= & \Box \mid \mathcal{C}.\texttt{<}Ts\texttt{>}m(es) \mid e.\texttt{<}Ts\texttt{>}m(es, \mathcal{C}, es') \\
& & \mid \mathcal{C}.[T]\texttt{<}Ts\texttt{>}m(es) \mid e.[T]\texttt{<}Ts\texttt{>}m(es, \mathcal{C}, es') \\
& & \mid \mathcal{C}.f \mid \texttt{new } C\texttt{<}Ts\texttt{>}(es, \mathcal{C}, es') \\
& & \mid \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>}(es, \mathcal{C}, es')\{Ms\} \\
& & \mid \texttt{if } (\mathcal{C} \texttt{ instanceof } T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \mid \texttt{if } (e_0 \texttt{ instanceof } T\ x)\ \{\ \mathcal{C}\ \}\ \texttt{else}\ \{\ e_2\ \} \\
& & \mid \texttt{if } (e_0 \texttt{ instanceof } T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ \mathcal{C}\ \}
\end{array}
$$

An expression e' occurs in e when $e = \mathcal{C}[e']$, for an expression context $\mathcal{C}$.

```
inductive ExprCtx where
  | hole
  | callRecv (ctx : ExprCtx) (Ts : List Ty) (m :
     MethName) (es : List Expr)
  | callArg (e : Expr) (Ts : List Ty) (m : MethName) (
     es : List Expr)
      (ctx : ExprCtx) (es' : List Expr)
  | superRecv (ctx : ExprCtx) (T : Ty) (Ts : List Ty) (
     m : MethName) (es : List Expr)
  | superArg (e : Expr) (T : Ty) (Ts : List Ty) (m :
     MethName) (es : List Expr)
      (ctx : ExprCtx) (es' : List Expr)
  | field (ctx : ExprCtx) (f : FieldName)
  | newArg (C : ClassName) (Ts : List Ty) (es : List
     Expr) (ctx : ExprCtx) (es' : List Expr)
  | localArg (C : ClassName) (Fs : List Funnel) (C' :
     ClassName) (Ts : List Ty)
      (es : List Expr) (ctx : ExprCtx) (es' : List Expr
         ) (Ms : List Method)
  | ifCond (ctx : ExprCtx) (T : Ty) (x : VarName) (e1
     e2 : Expr)
  | ifThen (e0 : Expr) (T : Ty) (x : VarName) (ctx :
     ExprCtx) (e2 : Expr)
  | ifElse (e0 : Expr) (T : Ty) (x : VarName) (e1 :
     Expr) (ctx : ExprCtx)
def OccursIn (e' e : Expr) : Prop := exists ctx :
   ExprCtx, e = ctx.plug e'
```

Mapping. $\mathcal{C}[e']$ is `ctx.plug e'`.

K declared, $M \in K$, $M \in C$

Dec declared iff Dec ∈ Decls

new C_0<Fs> extends C'<Ts>(es){Ms} declared
  where K declared
    <_>_ m(_) { return e; } ∈ K
    new C_0<Fs> extends C'<Ts>(es){Ms} occurs in e

$M \in K$ iff $M \in Ms$
  where K ∈ {class _<_> extends _<_> { Ms _ }, new _<_> extends _<_>(_){Ms}}

$M \in C$ iff $M \in K$
  where K declared
    K ∈ {class C<_> extends _<_> { _ _ }, new C<_> extends _<_>(_){_}}

```
inductive Declared (d : Decls) : K -> Prop
  | dec {D} : List.Mem D d -> Declared d (.dec D)
  | loc {k Delta T0 m ps e C0 Fs C' Ts es Ms} :
      Declared d k -> List.Mem (.mk Delta T0 m ps e) k.
         Ms ->
      OccursIn (.newLocal C0 Fs C' Ts es Ms) e ->
      Declared d (.loc C0 Fs C' Ts es Ms)
def MethodIn (d : Decls) (M : Method) (C : ClassName) :
    Prop :=
  exists k, Declared d k /\ k.name = C /\ List.Mem M k.
     Ms
```

Mapping. K declared is `Declared d K`; $M \in K$ is `List.Mem M K.Ms`; $M \in C$ is `MethodIn d M C`.

generic(C), $|F|$

$\text{generic}(C) = C\texttt{<}\text{dom}(\Delta)\texttt{>}$
  where `class` C<Δ> `extends` _<_> { _ _ } declared
$\text{generic}(C) = C\texttt{<}\text{dom}(Fs)\texttt{>}$
  where `new` C<Fs> `extends` _<_>(_){_} declared
$|X\ \texttt{funnel}| = X \qquad |G| = G$

```
inductive Generic (d : Decls) : ClassName -> Ty -> Prop
  | dec {D} : Declared d (.dec D) -> Generic d D.C (.
     cls D.C (vars (dom D.bounds)))
  | loc {C Fs C' Ts es Ms} :
     Declared d (.loc C Fs C' Ts es Ms) -> Generic d C
        (.cls C (vars (domF Fs)))
def Funnel.abs : Funnel -> Ty
  | .ground G => G
  | .funnel X => .var X
def abs (Fs : List Funnel) : List Ty := Fs.map Funnel.
   abs
```

Mapping. $\text{generic}(C) = T$ is `Generic d C T`; $|F|$ is `Funnel.abs F` and $|Fs|$ is `abs Fs`.

Pairing

$\emptyset \overset{\Delta}{\Longleftrightarrow} \emptyset = \emptyset;\emptyset$
$X\ \texttt{funnel}, Fs \overset{\Delta}{\Longleftrightarrow} T, Ts = X\ \texttt{extends}\ \Delta(X), \Delta'; T, Ts'$
  where $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts'$
$G, Fs \overset{\Delta}{\Longleftrightarrow} G, Ts = Fs \overset{\Delta}{\Longleftrightarrow} Ts$

```
inductive Pairing (Delta : Bounds) : List Funnel ->
   List Ty -> Bounds -> List Ty -> Prop
  | nil : Pairing Delta [] [] [] []
  | funnel {X T T0 Fs Ts Delta' Ts'} :
     List.Mem { X := X, T := T } Delta -> Pairing
        Delta Fs Ts Delta' Ts' ->
     Pairing Delta (.funnel X :: Fs) (T0 :: Ts) ({ X
        := X, T := T } :: Delta') (T0 :: Ts')
  | ground {G Fs Ts Delta' Ts'} :
     Pairing Delta Fs Ts Delta' Ts' -> Pairing Delta (.
        ground G :: Fs) (G.ty :: Ts) Delta' Ts'
```

Mapping. $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts'$ is `Pairing Delta Fs Ts Delta' Ts'`; $\Delta(X) = T$ is `List.Mem { X := X, T := T } Delta`, unique by A5 and A7.

scope(C<Ts>)

$\text{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$
  where `class` C<Δ> `extends` _<_> { _ _ } declared
$\text{scope}(C_0\texttt{<}Ts\texttt{>}) = Fs \overset{\Delta;\Delta'}{\Longleftrightarrow} Ts$
  where <Δ'>_ m(_) { `return` e; } $\in C$
    `new` C_0<Fs> `extends` _<_>(_){_} occurs in e
    $\text{scope}(\text{generic}(C)) = \Delta; _$

```
inductive Scope (d : Decls) : Ty -> Bounds -> List Ty
   -> Prop
  | dec {D Ts} : Declared d (.dec D) -> Scope d (.cls D.
     C Ts) D.bounds Ts
  | loc {C Delta' T0 m ps e C0 Fs C' Ts' es Ms T Delta
     Ts'' Ts Delta0 Ts0} :
     MethodIn d (.mk Delta' T0 m ps e) C ->
     OccursIn (.newLocal C0 Fs C' Ts' es Ms) e ->
     Generic d C T -> Scope d T Delta Ts'' ->
     Pairing (Delta ++ Delta') Fs Ts Delta0 Ts0 ->
     Scope d (.cls C0 Ts) Delta0 Ts0
```

Mapping. $\text{scope}(T) = \Delta; Ts$ is `Scope d T Delta Ts`; $\text{scope}(\text{generic}(C))$ is `Generic d C T` followed by `Scope d T`.

typeOf(v)

$\text{typeOf}(v) = C\texttt{<}Gs\texttt{>}$
  where $v \in \{$`new` C<Gs>(_), `new` C<Gs> `extends` _<_>(_){_}$\}$

```
def typeOf : Value -> Ty
  | .new C Gs _ => .cls C Gs
  | .newLocal C Gs _ _ _ _ => .cls C Gs
```

841 ## A.4.4 Substitution and Renaming

Substitution of type variables

$$
\begin{array}{lcll}
X'\ \texttt{funnel}[Xs = Ts] & = & X'\ \texttt{funnel} & \text{if } X' \notin Xs \\
X_i\ \texttt{funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i & \\
G[Xs = Ts] & = & G & \\
X'[Xs = Ts] & = & X' & \text{if } X' \notin Xs \\
X_i[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i & \\
(\texttt{<}\Delta\texttt{>}T_0'\ m(\ldots)\ \{\ldots\})[Xs = Ts] & = & \ldots & \text{if } \mathrm{dom}(\Delta) \cap Xs = \emptyset
\end{array}
$$

The other cases go into every subterm, including the methods of a local class. Substitution of type variables is defined only when the type variables of the methods it enters are not in its domain. In the domain of a substitution, bounds Δ stand for $\mathrm{dom}(\Delta)$, and funnels Fs stand for their funnelled type variables; the ground funnels are ignored. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground.

842

```
abbrev TSubst := List (Prod TVarName Ty)
def Ty.subst (sigma : TSubst) : Ty -> Ty
  | .var X => (sigma.lookup X).getD (.var X)
  | .cls C Ts => .cls C (Ty.substList sigma Ts)
def bind {A : Type} (Delta : Bounds) (as : List A) :
   List (Prod TVarName A) := (dom Delta).zip as
def bindF (Fs : List Funnel) (Ts : List Ty) : TSubst :=
  (Fs.zip Ts).filterMap fun
    | (.funnel X, T) => some (X, T)
    | (.ground _, _) => none
abbrev GSubst := List (Prod TVarName GTy)
def Funnel.gsubst (sigma : GSubst) : Funnel -> Funnel
  | .ground G => .ground G
  | .funnel X =>
    match sigma.lookup X with
    | some G => .ground G
    | none => .funnel X
abbrev GSubstDefined (Xs : List TVarName) (e : Expr) :
   Prop :=
  forall X, List.Mem X e.tbinders -> Not (List.Mem X Xs
     )
```

Mapping. $[\Delta = Ts]$ is `bind Delta Ts`, with a premise that the lengths are the same; $[Fs = Ts]$ is `bindF Fs Ts`. The function `Expr.gsubst` is total; a rule that substitutes into an expression `e` has the premise `GSubstDefined Xs e`, which is the condition under which the substitution of the paper is defined. Substitution into expressions, `Expr.gsubst`, takes ground types, so X_i `funnel`$[\ldots = \ldots]$ is again a funnel.

Substitution of variables

$$
\begin{array}{l}
x_i[x_1, \ldots, x_n = v_1, \ldots, v_n] = v_i \\
(\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\})[xs = vs] \\
\quad = \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es[xs = vs])\{Ms[xs' = vs']\} \\
\quad \text{where } [xs' = vs'] \text{ is } [xs = vs] \text{ without the pair of } \texttt{this} \\
(\texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x')\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \})[xs = vs] = \ldots \quad \text{if } x' \notin xs \\
(\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1', \ldots, T_k\ x_k')\ \{\ldots\})[xs = vs] = \ldots \quad \text{if } x_1', \ldots, x_k' \notin xs
\end{array}
$$

The other cases go into every subterm. Substitution of variables is defined only when the variable x' of an **`if`** and the parameters $x_1', \ldots, x_k'$ of a method are not in xs.

843

```
abbrev VSubst := List (Prod VarName Value)
def Expr.vsubst (sigma : VSubst) : Expr -> Expr
  | .var x =>
    match sigma.lookup x with
    | some v => v
    | none => .var x
  ...
  | .newLocal C Fs C' Ts es Ms =>
    .newLocal C Fs C' Ts (Expr.vsubstList sigma es)
      (Method.vsubstList (sigma.filter fun p => p.1 !=
         this) Ms)
abbrev VSubstDefined (xs : List VarName) (e : Expr) :
   Prop :=
  forall x, List.Mem x e.vbinders -> Not (List.Mem x xs
     )
```

Mapping. $[xs' = vs']$ is `sigma.filter fun p => p.1 != this`. The function `Expr.vsubst` is total; a rule that substitutes into an expression `e` has the premise `VSubstDefined xs e`, which is the condition under which the substitution of the paper is defined.

---

fresh(e, e') and fresh(M, M')

The names bound in an expression e are the variable of every **if** occurring in e, and the names bound in every method of a local class occurring in e. The names bound in a method M are the type variables of its signature, its parameters, and the names bound in its body. fresh(e, e') holds when e' is e with the names bound in e renamed: e and e' are equal up to the names of their binders. fresh(M, M') holds when M' is M renamed in the same way. The variable **this** is never renamed.

```
inductive RenE : VRen -> TRen -> Expr -> Expr -> Prop
  | var {rv rt x} : RenE rv rt (.var x) (.var (x.ren rv
     ))
  ...
  | ifInst {rv rt e0 e0' T x y e1 e1' e2 e2'} :
     RenE rv rt e0 e0' -> RenE ((x, y) :: rv) rt e1 e1'
        -> RenE rv rt e2 e2' ->
     RenE rv rt (.ifInst e0 T x e1 e2) (.ifInst e0' (T.
        ren rt) y e1' e2')
inductive RenM : VRen -> TRen -> Method -> Method ->
   Prop
  | mk {rv rt Delta T0 m ps e e' Xs' xs'} :
     Xs'.length = Delta.length -> xs'.length = ps.
        length ->
     RenE ((ps.map VarDecl.x).zip xs' ++ rv) ((dom
        Delta).zip Xs' ++ rt) e e' ->
     RenM rv rt (.mk Delta T0 m ps e) (.mk ... (
        renamed by Xs' and xs') ... e')
def Fresh (e e' : Expr) : Prop := RenE [] [] e e' /\
   RenE [] [] e' e
def FreshM (M M' : Method) : Prop := RenM [] [] M M' /\
    RenM [] [] M' M
```

Mapping. `RenE rv rt e e'`: e' is e with every binder renamed to any name, and its bound occurrences renamed with it; `rv` and `rt` are the renamings of variables and type variables in scope. `RenE` alone allows a renaming that captures a free name; requiring the renaming in both directions excludes the capture of every free name; it does not exclude a binder renamed to **this**, which the implicit **this** of a method of a local class captures. A renaming that gives two nested binders the same name, or that names a binder **this**, is excluded by the other premises of the rules: `WFExpr` in `Reduces` (A7 and A8), `Nodup` and `VSubstDefined`. Thus, in a rule, `Fresh` is equality up to the names of binders, as in the paper.

A.4.5 Reduction

simpleFieldFound

$$\frac{\texttt{class } C\texttt{<_>} \texttt{ extends } \texttt{_<_>} \texttt{ \{ _ } T_1\ f_1\texttt{;} \ldots T_n\ f_n\texttt{; \}} \text{ declared} \qquad f_i \in f_1, \ldots, f_n}{\texttt{new } C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\texttt{.}f_i \to v_i}$$

```
inductive Step (d : Decls) : Expr -> Expr -> Prop
  | simpleFieldFound {D f} {Gs : List GTy} {vs1 vs :
     List Value} {v : Value} :
     Declared d (.dec D) ->
     vs1.length = D.Ss.length ->
     List.Mem (f, v) ((D.Ss.map Slot.f).zip vs1) ->
     Step d (.field (Value.new D.C Gs (vs1 ++ vs)) f)
        v
```

Mapping. $v_1, \ldots, v_n$ is `vs1`, as long as the slots; f_i and v_i are a pair of `zip`.

simpleFieldSuper

$$\frac{\texttt{class } C\texttt{<}\Delta\texttt{>} \texttt{ extends } C'\texttt{<}Ts\texttt{>} \texttt{ \{ _ } T_1\ f_1\texttt{;} \ldots T_n\ f_n\texttt{; \}} \text{ declared} \qquad f \notin f_1, \ldots, f_n \qquad C \neq \texttt{Object}}{\texttt{new } C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\texttt{.}f \to \texttt{new } C'\texttt{<}Ts[\Delta = Gs]\texttt{>}(vs)\texttt{.}f}$$

```
  | simpleFieldSuper {D f} {Gs : List GTy} {vs1 vs :
     List Value} :
     Declared d (.dec D) ->
     vs1.length = D.Ss.length ->
     Not (List.Mem f (D.Ss.map Slot.f)) ->
     Not (D.C = Object) ->
     Gs.length = D.bounds.length ->
     Step d (.field (Value.new D.C Gs (vs1 ++ vs)) f)
       (.field (.new D.C' (Ty.substList (bind D.bounds
           (Gs : List Ty)) D.Ts) vs) f)
```

innerFieldSuper

$$\frac{}{\texttt{new } C\texttt{<}Gs\texttt{> extends } C'\texttt{<}Gs'\texttt{>}(vs)\texttt{\{}Ms\texttt{\}}\texttt{.}f \to \texttt{new } C'\texttt{<}Gs'\texttt{>}(vs)\texttt{.}f}$$

```
  | innerFieldSuper {C C' Ms f} {Gs Gs' : List GTy} {vs
     : List Value} :
     Step d (.field (Value.newLocal C Gs C' Gs' vs Ms)
        f) (.field (Value.new C' Gs' vs) f)
```

ifThen

$$\frac{\emptyset \vdash \mathsf{typeOf}(v) <: C'\texttt{<_>} \qquad \mathsf{fresh}(e_1, e_1')}{\texttt{if (}v\texttt{ instanceof }C'\texttt{<}Ts\texttt{> }x\texttt{) \{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}} \to e_1'[x = v]}$$

```
| ifThen {C' Ts Ts' x e1 e1' e2} {v : Value} :
    Sub d [] (typeOf v) (.cls C' Ts') ->
    Fresh e1 e1' ->
    VSubstDefined [x] e1' ->
    Step d (.ifInst v (.cls C' Ts) x e1 e2) (e1'.
       vsubst [(x, v)])
```

Mapping. The _ in C'<_> is the implicit `Ts'`.

ifElse

$$\frac{\emptyset \not\vdash \mathsf{typeOf}(v) <: C'\texttt{<_>}}{\texttt{if (}v\texttt{ instanceof }C'\texttt{<}Ts\texttt{> }x\texttt{) \{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}} \to e_2}$$

```
| ifElse {C' Ts x e1 e2} {v : Value} :
    Not (exists Ts', Sub d [] (typeOf v) (.cls C' Ts')
       ) ->
    Step d (.ifInst v (.cls C' Ts) x e1 e2) e2
```

simpleAddSuper

$$\frac{v = \texttt{new }C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs)}$$

```
| simpleAddSuper {C m} {v : Value} {Gs Gs' : List GTy
   } {vs0 vs : List Value} :
    v = Value.new C Gs vs0 ->
    Step d (.call v Gs' m vs) (.superCall v (.cls C
       Gs) Gs' m vs)
```

simpleFound

$$\frac{\begin{array}{c}\texttt{class }C\texttt{<}\Delta\texttt{> extends _<_> \{ }Ms\texttt{ _ \}}\ \mathsf{declared} \\ M \in Ms \\ \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{ return }e_0\texttt{; \}}) \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

```
| simpleFound {D M m Delta' T0 ps e0 e} {v0 : Value}
   {Gs Gs' : List GTy} {vs : List Value} :
    Declared d (.dec D) ->
    List.Mem M D.Ms ->
    FreshM M (.mk Delta' T0 m ps e0) ->
    Gs.length = D.bounds.length -> Gs'.length = Delta
       '.length ->
    (dom D.bounds ++ dom Delta').Nodup ->
    (this :: ps.map VarDecl.x).Nodup ->
    ps.length = vs.length ->
    GSubstDefined (dom D.bounds ++ dom Delta') e0 ->
    VSubstDefined (this :: ps.map VarDecl.x) e0 ->
    e = (e0.gsubst (bind (D.bounds ++ Delta') (Gs ++
       Gs'))).vsubst
      ((this :: ps.map VarDecl.x).zip (v0 :: vs)) ->
    Step d (.superCall v0 (.cls D.C Gs) Gs' m vs) e
```

Mapping. $x_1, \ldots, x_n$ is `ps.map VarDecl.x`, as many as the arguments `vs`.

innerFound

$$\frac{\begin{array}{c}v_0 = \texttt{new _<_> extends _<_>(_)\{}Ms\texttt{\}} \\ M \in Ms \\ \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{ return }e_0\texttt{; \}}) \\ e = e_0[\Delta' = Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

```
| innerFound {C C' Ms M m Delta' T0 ps e0 e} {v0 :
   Value} {Gs Gs'' Gs' : List GTy} {vs0 vs : List
   Value} :
    v0 = Value.newLocal C Gs C' Gs'' vs0 Ms ->
    List.Mem M Ms ->
    FreshM M (.mk Delta' T0 m ps e0) ->
    Gs'.length = Delta'.length ->
    ps.length = vs.length ->
    (dom Delta').Nodup ->
    (this :: ps.map VarDecl.x).Nodup ->
    GSubstDefined (dom Delta') e0 ->
    VSubstDefined (this :: ps.map VarDecl.x) e0 ->
    e = (e0.gsubst (bind Delta' Gs')).vsubst ((this ::
        ps.map VarDecl.x).zip (v0 :: vs)) ->
    Step d (.call v0 Gs' m vs) e
```

innerSuper

$$\frac{v_0 = \texttt{new}\ _\texttt{<}_\texttt{>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>}(_)\{Ms\} \qquad m \notin Ms}{v_0.\texttt{<}Gs'\texttt{>}m(v_1,\ldots,v_n) \to v_0.[C\texttt{<}Gs\texttt{>}]\texttt{<}Gs'\texttt{>}m(v_1,\ldots,v_n)}$$

```
| innerSuper {C C' Ms m} {v0 : Value} {Gs0 Gs Gs' :
   List GTy} {vs0 vs : List Value} :
    v0 = Value.newLocal C Gs0 C' Gs vs0 Ms ->
    Not (List.Mem m (Ms.map Method.name)) ->
    Step d (.call v0 Gs' m vs) (.superCall v0 (.cls C'
        Gs) Gs' m vs)
```

Mapping. The superclass C of the paper is `C'` in Lean.

searchSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ _\ \}\ \texttt{declared} \qquad m \notin Ms \qquad C \neq \texttt{Object}}{v.[C\texttt{<}Gs\texttt{>}]\texttt{<}Gs'\texttt{>}m(vs) \to v.[C'\texttt{<}Ts[\Delta = Gs]\texttt{>}]\texttt{<}Gs'\texttt{>}m(vs)}$$

```
| searchSuper {D m} {v : Value} {Gs Gs' : List GTy} {
   vs : List Value} :
    Declared d (.dec D) ->
    Not (List.Mem m (D.Ms.map Method.name)) ->
    Not (D.C = Object) ->
    Gs.length = D.bounds.length ->
    Step d (.superCall v (.cls D.C Gs) Gs' m vs)
      (.superCall v (.cls D.C' (Ty.substList (bind D.
         bounds (Gs : List Ty)) D.Ts)) Gs' m vs)
```

ctx, $\to$ and $\to^*$

$$\frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

An instance of a reduction rule $e \to e'$ exists only when the right side e' is an expression (Section 8). $e_0 \to^* e$: zero or more steps.

```
  | ctx {E : ECtx} {e e'} :
     Step d e e' -> Step d (E.plug e) (E.plug e')
def Reduces (d : Decls) (e e' : Expr) : Prop := Step d
   e e' /\ WFExpr e'
inductive Steps (d : Decls) : Expr -> Expr -> Prop
  | refl {e} : Steps d e e
  | step {e e' e''} : Reduces d e e' -> Steps d e' e''
     -> Steps d e e''
def IsValue (e : Expr) : Prop := exists v : Value, e =
   v
```

Mapping. $e \to e'$ is `Reduces d e e'`; $\to^*$ is `Steps d`; *e is of form v* is `IsValue e`.

A.4.6 Subtyping and Kinding

sub-refl, sub-var

$$\frac{}{\Delta \vdash T <: T} \qquad \frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

```
inductive Sub (d : Decls) (Delta : Bounds) : Ty -> Ty
   -> Prop
  | subRefl {T} : Sub d Delta T T
  | subVar {X T T'} :
     List.Mem { X := X, T := T } Delta -> Sub d Delta
        T T' -> Sub d Delta (.var X) T'
```

Mapping. $\Delta \vdash T <: T'$ is `Sub d Delta T T'`.

sub-inherit

$$\frac{\texttt{class}\ C\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ _\ \}\ \texttt{declared} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}[\Delta_0 = Ts_0] <: T'}{\Delta \vdash C\texttt{<}Ts_0\texttt{>} <: T'}$$

```
| subInherit {D Ts0 T'} :
    Declared d (.dec D) ->
    Ts0.length = D.bounds.length ->
    Sub d Delta ((Ty.cls D.C' D.Ts).subst (bind D.
       bounds Ts0)) T' ->
    Sub d Delta (.cls D.C Ts0) T'
```

Mapping. Δ_0 is `D.bounds`.

sub-inherit-local

$$\frac{\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(_)\{_\}\ \texttt{declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T'}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T'}$$

```
| subInheritLocal {C Fs C' Ts' es Ms Ts T'} :
    Declared d (.loc C Fs C' Ts' es Ms) ->
    Ts.length = Fs.length ->
    Sub d Delta ((Ty.cls C' Ts').subst (bindF Fs Ts))
        T' ->
    Sub d Delta (.cls C Ts) T'
```

wf-var, wf-class

$$\frac{X \texttt{ extends } _ \in \Delta}{\Delta \vdash X \text{ Ok}}$$

$$\frac{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0 \qquad \Delta \vdash Ts \text{ Ok} \qquad \Delta \vdash Ts_0 <: \mathrm{cod}(\Delta_0)[\Delta_0 = Ts_0]}{\Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok}}$$

```
inductive Kind (d : Decls) (Delta : Bounds) : Ty ->
   Prop
  | wfVar {X T} : List.Mem { X := X, T := T } Delta ->
     Kind d Delta (.var X)
  | wfClass {C Ts Delta0 Ts0} :
     Scope d (.cls C Ts) Delta0 Ts0 ->
     (forall T, List.Mem T Ts -> Kind d Delta T) ->
     Ts0.length = Delta0.length ->
     SubSeq d Delta Ts0 (Ty.substList (bind Delta0 Ts0
        ) (cod Delta0)) ->
     Kind d Delta (.cls C Ts)
```

A.4.7 Checked Cast

cast-up, cast-var

$$\frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'} \qquad \frac{X \texttt{ extends } T \in \Delta \quad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

```
inductive Cast (d : Decls) (Delta : Bounds) : Ty -> Ty
   -> Prop
  | castUp {T T'} : Sub d Delta T T' -> Cast d Delta T
     T'
  | castVar {X T T'} :
     List.Mem { X := X, T := T } Delta -> Cast d Delta
        T T' -> Cast d Delta (.var X) T'
```

Mapping. $\Delta \vdash T \rightsquigarrow T'$ is `Cast d Delta T T'`.

cast-down

$$\frac{\Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>} \qquad \mathrm{generic}(C) = C\texttt{<}Xs\texttt{>} \qquad \emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>} \qquad Xs \subseteq \mathsf{FTV}(Ts')}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

```
  | castDown {C Ts C' Ts0 Xs Ts'} :
     Sub d Delta (.cls C Ts) (.cls C' Ts0) ->
     Generic d C (.cls C (vars Xs)) ->
     Sub d [] (.cls C (vars Xs)) (.cls C' Ts') ->
     (forall X, List.Mem X Xs -> List.Mem X (Ty.
        ftvList Ts')) ->
     Cast d Delta (.cls C' Ts0) (.cls C Ts)
```

Mapping. $C\texttt{<}Xs\texttt{>}$ is `.cls C (vars Xs)`.

A.4.8 Typing

subs, var, new-object

$$\frac{\Delta; \Gamma \vdash e : T_1 \qquad \Delta \vdash T_2 \text{ Ok} \qquad \Delta \vdash T_1 <: T_2}{\Delta; \Gamma \vdash e : T_2}$$

$$\frac{}{\Delta; \Gamma \vdash x : \Gamma(x)}$$

$$\frac{}{\Delta; \Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

```
inductive Typing (d : Decls) : Bounds -> Gamma -> Expr
   -> Ty -> Prop
  | subs {Delta gamma e T1 T2} :
     Typing d Delta gamma e T1 -> Kind d Delta T2 ->
        Sub d Delta T1 T2 ->
     Typing d Delta gamma e T2
  | var {Delta gamma x T} :
     gamma.lookup x = some T -> Typing d Delta gamma (.
        var x) T
  | newObject {Delta gamma} :
     Typing d Delta gamma (.new Object [] []) (.cls
        Object [])
```

Mapping. $\Delta; \Gamma \vdash e : T$ is `Typing d Delta gamma e T`.

declField

$$\frac{\Delta; \Gamma \vdash e : C\texttt{<}Ts'\texttt{>} \qquad \texttt{class } C\texttt{<}\Delta'\texttt{> extends } _\texttt{<}_\texttt{> \{ } _ \; Ss \texttt{ \} declared} \qquad T\ f; \in Ss}{\Delta; \Gamma \vdash e.f : T[\Delta' = Ts']}$$

```
  | declField {Delta gamma e D Ts' T f} :
     Typing d Delta gamma e (.cls D.C Ts') ->
     Declared d (.dec D) ->
     List.Mem { T := T, f := f } D.Ss ->
     Ts'.length = D.bounds.length ->
     Typing d Delta gamma (.field e f) (T.subst (bind
        D.bounds Ts'))
```

Mapping. Δ' is `D.bounds`; Ss is `D.Ss`.

call

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts_0\texttt{>} \\ \mathsf{scope}(C_0\texttt{<}Ts_0\texttt{>}) = \Delta_0; Ts_2 \\ \texttt{<}\Delta_1\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ _\ \} \in C_0 \\ \forall i \in 0..n.\ T_i' = T_i[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta \vdash Ts_1\ \mathrm{Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0.\texttt{<}Ts_1\texttt{>}m(e_1, \ldots, e_n) : T_0'}$$

```
| call {Delta gamma e0 C0 Ts0 Delta0 Ts2 Delta1 T0 m
   ps body Ts1 es T0' Ts'} :
    Typing d Delta gamma e0 (.cls C0 Ts0) ->
    Scope d (.cls C0 Ts0) Delta0 Ts2 ->
    MethodIn d (.mk Delta1 T0 m ps body) C0 ->
    Ts2.length = Delta0.length -> Ts1.length = Delta1.
       length ->
    T0' :: Ts' = Ty.substList (bind (Delta0 ++ Delta1
       ) (Ts2 ++ Ts1))
      (T0 :: ps.map VarDecl.T) ->
    KindSeq d Delta Ts1 ->
    SubSeq d Delta Ts1
      (Ty.substList (bind (Delta0 ++ Delta1) (Ts2 ++
         Ts1)) (cod Delta1)) ->
    TypingSeq d Delta gamma es Ts' ->
    Typing d Delta gamma (.call e0 Ts1 m es) T0'
```

Mapping. $T_0', \ldots, T_n'$ is `T0' :: Ts'`.

call-lb-runtime

$$\frac{\begin{array}{c}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C_0\texttt{<}Ts_0\texttt{>} \\ \texttt{class}\ C_0\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \{\ _\ _\ \}\ \text{declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ _\ \} \in C_0 \\ \forall i \in 0..n.\ T_i' = T_i[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \texttt{class}\ C\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \{\ _\ _\ \}\ \text{declared} \\ \Delta \vdash C\texttt{<}Ts\texttt{>}\ \mathrm{Ok} \qquad \Delta \vdash Ts_1\ \mathrm{Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \Delta;\Gamma \vdash e_0, \ldots, e_n : C\texttt{<}Ts\texttt{>}, T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0.\,[C\texttt{<}Ts\texttt{>}]\texttt{<}Ts_1\texttt{>}m(e_1, \ldots, e_n) : T_0'}$$

```
| callLbRuntime {Delta gamma e0 D Ts D0 Ts0 Delta1 T0
    m ps body Ts1 es T0' Ts'} :
    Sub d Delta (.cls D.C Ts) (.cls D0.C Ts0) ->
    Declared d (.dec D0) ->
    MethodIn d (.mk Delta1 T0 m ps body) D0.C ->
    Ts0.length = D0.bounds.length -> Ts1.length =
       Delta1.length ->
    T0' :: Ts' = Ty.substList (bind (D0.bounds ++
       Delta1) (Ts0 ++ Ts1))
      (T0 :: ps.map VarDecl.T) ->
    Declared d (.dec D) ->
    Kind d Delta (.cls D.C Ts) ->
    KindSeq d Delta Ts1 ->
    SubSeq d Delta Ts1
      (Ty.substList (bind (D0.bounds ++ Delta1) (Ts0
         ++ Ts1)) (cod Delta1)) ->
    TypingSeq d Delta gamma (e0 :: es) (.cls D.C Ts ::
        Ts') ->
    Typing d Delta gamma (.superCall e0 (.cls D.C Ts)
        Ts1 m es) T0'
```

Mapping. C is `D.C` and C_0 is `D0.C`.

new-top

$$\frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ T_1\ f_1; \ldots T_n\ f_n;\ \}\ \text{declared} \\ \Delta \vdash C\texttt{<}Ts'\texttt{>}\ \mathrm{Ok} \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n[\Delta' = Ts'] \\ \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts[\Delta' = Ts']\texttt{>}(es) : _\end{array}}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Ts'\texttt{>}(e_1, \ldots, e_n, es) : C\texttt{<}Ts'\texttt{>}}$$

```
| newTop {Delta gamma D Ts' es1 es2 T''} :
    Declared d (.dec D) ->
    Kind d Delta (.cls D.C Ts') ->
    Ts'.length = D.bounds.length ->
    TypingSeq d Delta gamma es1 (Ty.substList (bind D.
       bounds Ts') (D.Ss.map Slot.T)) ->
    Typing d Delta gamma (.new D.C' (Ty.substList (
       bind D.bounds Ts') D.Ts) es2) T'' ->
    Typing d Delta gamma (.new D.C Ts' (es1 ++ es2))
       (.cls D.C Ts')
```

Mapping. $e_1, \ldots, e_n$ is `es1`; es is `es2`; the _ type is `T"`.

new-local

$$\frac{\begin{array}{c}\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>}(es)\{Ms\}\ \mathsf{consistent} \\ Fs \overset{\Delta}{\Longleftrightarrow} |Fs| = \Delta'; _ \\ \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \\ \Delta' \vdash C\texttt{<}|Fs|\texttt{>}\ \mathrm{Ok} \\ \Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \mathrm{Ok} \qquad \Delta; \Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>}(es) : _ \\ \Delta'; \Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}, C\texttt{<}|Fs|\texttt{>}\ \texttt{this} \vdash Ms\ \mathrm{Ok} \\ \mathsf{overrideOk}(\Delta', C_0\texttt{<}Ts\texttt{>}, Ms)\end{array}}{\Delta; \Gamma \vdash \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>}(es)\{Ms\} : C\texttt{<}|Fs|\texttt{>}}$$

For a set of type variables Xs, $\Gamma|_{Xs}$ keeps the variables $T\ x$ of Γ such that $\mathsf{FTV}(T) \subseteq Xs$. $\Gamma \setminus x$ is Γ without the variable x.

```
  | newLocal {Delta gamma C Fs C0 Ts es Ms Delta' Ts''
    T''} :
     Consistent d (.loc C Fs C0 Ts es Ms) ->
     Pairing Delta Fs (abs Fs) Delta' Ts'' ->
     (forall X, List.Mem X (Ty.ftvList (cod Delta')) ->
         List.Mem X (dom Delta')) ->
     Kind d Delta' (.cls C (abs Fs)) ->
     Kind d Delta' (.cls C0 Ts) ->
     Typing d Delta gamma (.new C0 Ts es) T'' ->
     (forall M, List.Mem M Ms ->
       MethodOk d Delta'
         (withoutThis (restrict gamma (Ty.ftvList (abs
           Fs))) ++
          [{ T := .cls C (abs Fs), x := this }]) M) ->
     OverrideOk d Delta' (.cls C0 Ts) Ms ->
     Typing d Delta gamma (.newLocal C Fs C0 Ts es Ms)
         (.cls C (abs Fs))
def restrict (gamma : Gamma) (Xs : List TVarName) :
   Gamma :=
  gamma.filter fun d => (Ty.ftv d.T).all fun X => Xs.
     contains X
def withoutThis (gamma : Gamma) : Gamma := gamma.filter
    fun d => d.x != this
```

Mapping. $\Gamma|_{Xs}$ is `restrict gamma Xs`; $\Gamma \setminus$ `this` is `withoutThis gamma`.

if-instanceof

$$\frac{\Delta; \Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T\ \mathrm{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \qquad \Delta; \Gamma, T\ x \vdash e_1 : T' \qquad \Delta; \Gamma \vdash e_2 : T'}{\Delta; \Gamma \vdash \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} : T'}$$

```
  | ifInstanceof {Delta gamma e0 T0 T x e1 e2 T'} :
     Typing d Delta gamma e0 T0 ->
     Kind d Delta T ->
     Cast d Delta T0 T ->
     Typing d Delta (gamma ++ [{ T := T, x := x }]) e1
        T' ->
     Typing d Delta gamma e2 T' ->
     Typing d Delta gamma (.ifInst e0 T x e1 e2) T'
```

Mapping. $\Gamma, T\ x$ is `gamma ++ [{ T := T, x := x }]`.

Judgements on sequences

A judgement applied to sequences stands for one application at every position, and requires the sequences to have the same length: $\Delta \vdash T_1, \ldots, T_n\ \mathrm{Ok}$ is $\Delta \vdash T_1\ \mathrm{Ok} \quad \ldots \quad \Delta \vdash T_n\ \mathrm{Ok}$; $\Delta \vdash T_1, \ldots, T_n <: T'_1, \ldots, T'_n$ is $\Delta \vdash T_1 <: T'_1 \quad \ldots \quad \Delta \vdash T_n <: T'_n$; $\Delta; \Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n$ is $\Delta; \Gamma \vdash e_1 : T_1 \quad \ldots \quad \Delta; \Gamma \vdash e_n : T_n$.

```
abbrev KindSeq (d : Decls) (Delta : Bounds) (Ts : List
   Ty) : Prop := forall T, List.Mem T Ts -> Kind d
   Delta T
abbrev SubSeq (d : Decls) (Delta : Bounds) : List Ty ->
    List Ty -> Prop := Seq2 (Sub d Delta)
inductive TypingSeq (d : Decls) : Bounds -> Gamma ->
   List Expr -> List Ty -> Prop
  | nil {Delta gamma} : TypingSeq d Delta gamma [] []
  | cons {Delta gamma e es T Ts} :
     Typing d Delta gamma e T -> TypingSeq d Delta
        gamma es Ts ->
     TypingSeq d Delta gamma (e :: es) (T :: Ts)
inductive Seq2 {A B : Type} (R : A -> B -> Prop) : List
    A -> List B -> Prop
  | nil : Seq2 R [] []
  | cons {a b as bs} : R a b -> Seq2 R as bs -> Seq2 R
    (a :: as) (b :: bs)
```

Mapping. `TypingSeq` is defined together with `Typing`. Inside the definition of `Kind`, rule WF-CLASS writes $\Delta \vdash Ts\ \mathrm{Ok}$ as a premise for every element, since `KindSeq` is defined after `Kind`.

A.4.9 Consistency and Overriding

K consistent

`new` C<Fs> `extends` C_0<Ts>(es){Ms} consistent
where `new` C<Fs'> `extends` C_0<Ts'>(_){Ms'} declared
$Ts = Ts'[Fs' = |Fs|]$
$\mathrm{dom}(Ms') = \mathrm{dom}(Ms)$
overrideOk(C<$|Fs|$>, Ms)

$\mathrm{dom}(Ms)$ is the set of the names of the methods in Ms.

```
def Consistent (d : Decls) : K -> Prop
  | .loc C Fs C0 Ts _ Ms =>
    exists Fs' Ts' es' Ms',
      Declared d (.loc C Fs' C0 Ts' es' Ms') /\
      Fs.length = Fs'.length /\
      Ts = Ty.substList (bindF Fs' (abs Fs)) Ts' /\
      (forall m, List.Mem m (Ms'.map Method.name) <->
         List.Mem m (Ms.map Method.name)) /\
      forall M, List.Mem M Ms -> OverrideOkM d (.cls C
         (abs Fs)) M
  | .dec _ => False
```

Mapping. overrideOk(C<$|Fs|$>, Ms) on the sequence Ms is one `OverrideOkM` per method.

---

overrideOk(Δ, T, Ms)

overrideOk(Δ, T, Ms) iff overrideOk(C<Ts>, M)
for every $M \in Ms$ and every C<Ts> such that $\Delta \vdash T <: C$<Ts>

```
def OverrideOk (d : Decls) (Delta : Bounds) (T : Ty) (
   Ms : List Method) : Prop :=
  forall M C Ts, List.Mem M Ms -> Sub d Delta T (.cls C
      Ts) -> OverrideOkM d (.cls C Ts) M
```

Mapping. overrideOk(T, M) is `OverrideOkM d T M`.

---

overrideOk(T, M)

overrideOk(C<Ts>, <_>_ m(_) { _ })
where `class` C<_> `extends` _<_> { Ms _ } declared
$m \notin Ms$

overrideOk(C<Ts>, <Δ'>T_0' m(T_1' x_1, …, T_n' x_n) { _ })
where `class` C<Δ> `extends` _<_> { Ms _ } declared
<Δ''>T_0 m(T_1 x_1', …, T_n x_n') { _ } $\in Ms$
$\mathrm{cod}(\Delta'')[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0, \ldots, T_n[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

overrideOk(C<Ts>, <Δ'>T_0' m(T_1' x_1, …, T_n' x_n) { _ })
where `new` C<Fs> `extends` _<_>(_){Ms} declared
<Δ''>T_0 m(T_1 x_1', …, T_n x_n') { _ } $\in Ms$
$\mathrm{cod}(\Delta'')[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0, \ldots, T_n[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

```
inductive OverrideOkM (d : Decls) : Ty -> Method ->
   Prop
  | absent {D Ts Delta' T0' m ps' e'} :
      Declared d (.dec D) ->
      Not (List.Mem m (D.Ms.map Method.name)) ->
      OverrideOkM d (.cls D.C Ts) (.mk Delta' T0' m ps'
          e')
  | dec {D Ts Delta' T0' m ps' e' Delta'' T0 ps e} :
      Declared d (.dec D) ->
      List.Mem (Method.mk Delta'' T0 m ps e) D.Ms ->
      Ts.length = D.bounds.length -> Delta'.length =
         Delta''.length ->
      Ty.substList (bind (D.bounds ++ Delta'') (Ts ++
         vars (dom Delta')))
        (cod Delta'') = cod Delta' ->
      Ty.substList (bind (D.bounds ++ Delta'') (Ts ++
         vars (dom Delta')))
        (T0 :: ps.map VarDecl.T) = T0' :: ps'.map
           VarDecl.T ->
      OverrideOkM d (.cls D.C Ts) (.mk Delta' T0' m ps'
          e')
  | loc {C Fs C0 Ts0 es Ms Ts Delta' T0' m ps' e' Delta
     '' T0 ps e} :
      Declared d (.loc C Fs C0 Ts0 es Ms) ->
      List.Mem (Method.mk Delta'' T0 m ps e) Ms ->
      Ts.length = Fs.length -> Delta'.length = Delta''.
         length ->
      Ty.substList (bindF Fs Ts ++ bind Delta'' (vars (
         dom Delta'))))
        (cod Delta'') = cod Delta' ->
      Ty.substList (bindF Fs Ts ++ bind Delta'' (vars (
         dom Delta'))))
        (T0 :: ps.map VarDecl.T) = T0' :: ps'.map
           VarDecl.T ->
      OverrideOkM d (.cls C Ts) (.mk Delta' T0' m ps' e
         ')
```

Mapping. One constructor per case of the paper. Comparing the lists $T_0, \ldots, T_n$ and $T_0', \ldots, T_n'$ also requires the same number of parameters, as the paper does by writing n in both methods.

A.4.10 Methods, Classes and Programs

method-ok

$$\frac{\Delta, \Delta' \vdash \mathrm{cod}(\Delta')\ \mathrm{Ok} \qquad \Delta, \Delta' \vdash T_0, \ldots, T_n\ \mathrm{Ok} \qquad \Delta, \Delta'; \Gamma, T_1\ x_1, \ldots, T_n\ x_n \vdash e : T_0}{\Delta; \Gamma \vdash \texttt{<}\Delta'\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e\texttt{;}\ \}\ \mathrm{Ok}}$$

```
inductive MethodOk (d : Decls) : Bounds -> Gamma ->
   Method -> Prop
 | methodOk {Delta gamma Delta' T0 m ps e} :
    KindSeq d (Delta ++ Delta') (cod Delta') ->
    KindSeq d (Delta ++ Delta') (T0 :: ps.map VarDecl.
       T) ->
    Typing d (Delta ++ Delta') (gamma ++ ps) e T0 ->
    MethodOk d Delta gamma (.mk Delta' T0 m ps e)
```

Mapping. The parameters $T_1\ x_1, \ldots, T_n\ x_n$ are a `Gamma`, so $\Gamma, T_1\ x_1, \ldots$ is `gamma ++ ps`.

class-ok

$$\frac{\Delta \vdash \mathrm{cod}(\Delta)\ \mathrm{Ok} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}, T_1, \ldots, T_n\ \mathrm{Ok} \qquad \Delta; C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}\ \texttt{this} \vdash Ms\ \mathrm{Ok} \qquad \mathrm{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \}\ \mathrm{Ok}}$$

```
def ClassOk (d : Decls) (D : Dec) : Prop :=
  KindSeq d D.bounds (cod D.bounds) /\
  KindSeq d D.bounds (Ty.cls D.C' D.Ts :: D.Ss.map Slot.
     T) /\
  (forall M, List.Mem M D.Ms ->
    MethodOk d D.bounds [{ T := .cls D.C (vars (dom D.
       bounds)), x := this }] M) /\
  OverrideOk d D.bounds (.cls D.C' D.Ts) D.Ms
```

Mapping. A rule with one conclusion and no recursion is a definition: the four premises are the four conjuncts.

prog-ok

$$\frac{\forall \mathrm{Dec} \in \mathrm{Decls}.\ \mathrm{Dec}\ \mathrm{Ok} \qquad \emptyset; \emptyset \vdash e : _}{\vdash \mathrm{Decls}\ e\ \mathrm{Ok}}$$

```
def DeclsOk (d : Decls) : Prop := forall D, List.Mem D
   d -> ClassOk d D
def ProgOk (d : Decls) (e : Expr) : Prop :=
  DeclsOk d /\ exists T, Typing d [] [] e T
```

Mapping. `DeclsOk d` is the first premise; it is also the hypothesis of Type Soundness.

A.5 Statements and High Level Proofs

The fixed statements are the two theorems of Section 5, Type Soundness and Determinism, on the high level:

```
theorem typeSoundness {d e0 e T} (hwf : WFDecls d) (hok : DeclsOk d) (he0 : WFExpr e0)
    (ht : Typing d [] [] e0 T) (hs : Steps d e0 e) :
    Typing d [] [] e T /\ (IsValue e \/ exists e', Reduces d e e')
theorem determinism {d e e1 e2} (hwf : WFDecls d) (he : WFExpr e) (h1 : Reduces d e e1)
    (h2 : Reduces d e e2) : Fresh e1 e2
```

`hok` says that Dec Ok for every Dec ∈ Decls; `hwf` says that Decls satisfies A1-A11, and `he0` says that the initial expression satisfies A3-A8 and A12, as every program of the paper does; in `determinism`, `he` says that the expression that reduces satisfies A3-A8 and A12.

The proofs are in `lean/Funnelling/Proof.lean`, and they can take any structure; `determinism` is stated there with `sorry`. For `typeSoundness`, the two structures below are both acceptable:

- **Divide first, then translate.** Type Soundness follows from progress and preservation on the high level, by induction on `Steps`. Progress and preservation on the high level follow from progress and preservation on the low level, through adequacy lemmas: the translation preserves values, well-formedness, Dec Ok and typing, and the two reductions simulate each other. This is the structure of `lean/Funnelling/Proof.lean` now.
- **Translate first, then divide.** Type Soundness is stated on the low level and proved there, from progress and preservation on the low level. The whole statement is then transported to the high level, through a simulation between the reductions of the two levels. This structure is needed when the high level is sound, but progress or preservation do not hold on the high level as stated, for example because of the names chosen by fresh.

In both structures, progress, preservation and the adequacy lemmas are components of the proof, not statements of the paper. `lean/Funnelling/Proof.lean` now uses these adequacy lemmas, stated with `sorry`:

```
theorem isValue_tr {e : High.Expr} : High.IsValue e <-> (trExpr e).isValue = true
theorem wfExpr_tr {e : High.Expr} (he : High.WFExpr e) : Low.wfExpr (trExpr e) = true
theorem declsOk_tr {d : High.Decls} (hwf : High.WFDecls d) (hok : High.DeclsOk d) :
    Low.DeclsOk (trDecls d)
theorem typing_tr {d : High.Decls} {e : High.Expr} {T : High.Ty} (hwf : High.WFDecls d) :
    High.Typing d [] [] e T <->
      Low.Typing (trDecls d) [] [] (trExpr e) (trTy T)
theorem step_tr {d : High.Decls} {e e' : High.Expr} (hwf : High.WFDecls d) (he : High.WFExpr e)
    (hs : High.Reduces d e e') : Low.Step (trDecls d) (trExpr e) (trExpr e')
theorem step_back {d : High.Decls} {e : High.Expr} {e'' : Low.Expr} (hwf : High.WFDecls d)
    (he : High.WFExpr e) (hs : Low.Step (trDecls d) (trExpr e) e'') :
    exists e', High.Reduces d e e'
```

The low level, the translation and these lemmas can change together; what must not change is the high level and the statements of `typeSoundness` and `determinism`. The planned proofs of progress and preservation, on whichever level they are proved, are:

1. **Basic lemmas.** `lookup` on a substitution built by `zip`; by A1, declared, scope and generic have one result per class name.
2. **Renaming.** A renaming by `Fresh` keeps typing and kinding under $\Delta;\Gamma$ when the renamed expression binds no name of $\mathrm{dom}(\Delta)$ or $\mathrm{dom}(\Gamma)$ and satisfies A7 and A8; in the rules, `VSubstDefined`, `GSubstDefined` and `Nodup` give the first condition; `Fresh` alone does not keep A7 and A8 (W26, W27).
3. **Weakening** of Γ and Δ, for typing, kinding and subtyping.
4. **Substitution of type variables.** If Δ, Δ' types e, and ground types Gs satisfy the bounds Δ', then $e[\Delta' = Gs]$ is typed under Δ with the substituted type. Kinding and subtyping are preserved, but the checked cast is not: in rule CAST-VAR, after X becomes G, the source G may not reach the target through CAST-UP or CAST-DOWN. For the typing of rule IF-INSTANCEOF the lemma uses rule SUBS to lift e_0 from G to the substituted bound of X, and applies there the substituted premise $\Delta \vdash T \rightsquigarrow T'$ of rule CAST-VAR, which ends in CAST-UP, CAST-VAR or CAST-DOWN.
5. **Substitution of variables.** Values are closed and typed under $\emptyset;\emptyset$, so substituting values for variables of the same types preserves typing; `this` is not substituted into the methods of a local class.
6. **Inversion** of typing for each form of expression, through rule SUBS.
7. **Canonical forms.** A value of type $C\texttt{<}Ts\texttt{>}$ has a class that is a subclass of C.
8. **Copies of a local class.** By consistent, the methods of a copy have the signatures of the declaration with the funnelled type variables substituted; so rules CALL and SUB-INHERIT-LOCAL, which read the declaration, agree with rule INNERFOUND, which runs the copy.
9. **Checked cast.** If $\emptyset \vdash \mathsf{typeOf}(v) <: C\texttt{<_>}$, v has type T_0 and $\emptyset \vdash T_0 \rightsquigarrow C\texttt{<}Ts\texttt{>}$, then v has type $C\texttt{<}Ts\texttt{>}$. The proof uses that the chain of superclasses is unique and that $Xs \subseteq \mathsf{FTV}(Ts')$ fixes Ts.
10. **Well-formedness of the result.** A step whose fresh names avoid the names of the program keeps A7; by Dec Ok, every variable and every type variable of a method body is bound, so a step keeps A12; the other requirements are local; progress needs it, since a step of the paper reduces to a well-formed expression.
11. **Progress** by induction on typing, with canonical forms; a call terminates its search at a class declaring the method, before `Object`, since typing found the method in a superclass.
12. **Preservation** by induction on the reduction, with the lemmas above.

A.6 Validation Examples

`lean/Funnelling/Examples.lean` defines the class tables below and the examples, on the high level. A Lean theorem marked *proved* is accepted by Lean, with no axiom other than the standard ones; a theorem marked *stated* has `sorry`: a positive one needs a derivation, and a negative one needs the inversion lemmas of Section A.5. The core language has no strings, no lambdas, no `switch` and no casts. The examples of Section 2 use `Fun<X,Y>` for `Function<X,Y>`, and its subclasses `AToB` and `AsB` for the lambdas of the two calls of `m`. `AsB` plays the lambda `s->s`: its method `apply` uses its argument as a `B<>`, so a failing cast to `String` becomes the call `new A<>().onlyB()`, which reduces by SIMPLEADDSUPER and SEARCHSUPER to `new A<>().[Object<>]onlyB()`, which is stuck. Examples 2 and 3 differ only in the cast, so in the core language they are one program; Examples 4 and 5 differ from Example 3 only in `switch` and in the bytecode, so they have no counterpart.

```
class Object<> extends Object<>{}
class A<> extends Object<>{}
class B<> extends Object<>{ B<> onlyB(){ return new B<>(); } }
class Foo<X extends Object<>> extends Object<>{ X x(){ return this.x(); } }
class Fun<X extends Object<>,Y extends Object<>> extends Object<>{ Y apply(X x){
    return this.apply(x); } }
class AToB<> extends Fun<A<>,B<>>{ B<> apply(A<> a){ return new B<>(); } }
class AsB<> extends Fun<B<>,B<>>{ B<> apply(B<> b){ return b.onlyB(); } }
// d1: Object, A, B, and
class Box<X extends Object<>> extends Object<>{ X val; X get(){ return this.val; }
    }
class C1<> extends Object<>{ A<> m(){ return new A<>(); } }
class C2<> extends C1<>{}
// d2: Object, A, B, Foo, and
class Maker<> extends Object<>{ <Y extends Object<>> Foo<Y> mk(Y y){ return
  new MyFoo<Y funnel> extends Foo<Y>(){ Y x(){ return y; } }; } }
// d3, Example 1: Object, A, B, Foo, and
class Broken<> extends Object<>{ <X extends Object<>> Foo<X> m(Object<> other, X x)
    { return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; } }
// d4: Object, A, B, Foo, and
class Leaker<> extends Object<>{
  <Y extends Object<>> MyFoo2<> mk2(Y y){ return new MyFoo2<> extends Foo<Y>(){ Y x
      (){ return y; } }; }
  <Y extends Object<>> Foo<Y> leak(MyFoo2<> z){ return z; } }
// d4F: Object, A, B, Foo, and
class LeakerF<> extends Object<>{
  <Y extends Object<>> MyFoo2F<Y> mk2(Y y){ return new MyFoo2F<Y funnel> extends
      Foo<Y>(){ Y x(){ return y; } }; }
  <Y extends Object<>> Foo<Y> leak(MyFoo2F<Y> z){ return z; } }
// d5, Example 1 funnelled: Object, A, B, Foo, and
class Fixed<> extends Object<>{ <X extends Object<>> Foo<X> mk(Foo<X> other, X x){
    return
  if (other instanceof MyFoo<X> w){ w }
  else { new MyFoo<X funnel> extends Foo<X>(){ X x(){ return x; } } }; } }
// d6, Examples 2 and 3: Object, A, B, Fun, AToB, AsB, and
class Broken2<> extends Object<>{ <X extends Object<>> Object<> m(Object<> other, X
     x, Fun<X,B<>> f){ return
  if (other instanceof MyFoo2<> local){ f.apply(local.x()) }
```

```
  else { new MyFoo2<> extends Object<>(){ X x(){ return x; } } }; } }
// d6F: Object, A, B, Fun, AToB, AsB, and
class Holder<X extends Object<>> extends Object<>{}
class Fixed2<> extends Object<>{ <X extends Object<>> Object<> m(Holder<X> other, X
    x, Fun<X,B<>> f){ return
  if (other instanceof MyFoo3<X> local){ f.apply(local.x()) }
  else { new MyFoo3<X funnel> extends Holder<X>(){ X x(){ return x; } } }; } }
class Fixed2Obj<> extends Object<>{ <X extends Object<>> Object<> m(Object<> other,
    X x, Fun<X,B<>> f){ return
  if (other instanceof MyFoo4<X> local){ f.apply(local.x()) }
  else { new MyFoo4<X funnel> extends Object<>(){ X x(){ return x; } } }; } }
// d7, Example 6: Object, A, B, Fun, AToB, AsB, and
class Broken6<> extends Object<>{
  <X extends Object<>> Object<> m(Object<> other, X x, Fun<X,B<>> f){ return
    this.check(new MyFoo6<> extends Object<>(){ X v(){ return x; } B<> show(MyFoo6
        <> o){ return f.apply(o.v()); } },
     other); }
  Object<> check(MyFoo6<> mine, Object<> other){ return
    if (other instanceof MyFoo6<> local){ mine.show(local) } else { mine }; } }
// d8, Counterexample 7: Object, A, B, Fun, AToB, AsB, and
class Box7<X extends Object<>> extends Object<>{ Object<> m(Object<> other, X x,
    Fun<X,B<>> f){ return
  if (other instanceof MyFoo7<> local){ f.apply(local.x()) }
  else { new MyFoo7<> extends Object<>(){ X x(){ return x; } } }; } }
class Box7F<X extends Object<>> extends Object<>{ Object<> m(Object<> other, X x,
    Fun<X,B<>> f){ return
  if (other instanceof MyFoo7F<X> local){ f.apply(local.x()) }
  else { new MyFoo7F<X funnel> extends Object<>(){ X x(){ return x; } } }; } }
// d9, nested local classes: Object, A, B, Foo, and
class Nest<> extends Object<>{ <X extends Object<>> Foo<X> mk(X x){ return
  new Outer<X funnel> extends Foo<X>(){
    X x(){ return new Inner<X funnel> extends Foo<X>(){ X x(){ return x; } }.x(); }
        }; } }
// d10, ground funnels and funnels together: Object, A, B, Foo, and
class Mix<> extends Object<>{ <X extends Object<>> Foo<X> mk(X x){ return
  new Outer2<X funnel> extends Foo<X>(){ X x(){ return x; }
    <Y extends Object<>> Foo<Y> g(Y y){ return new Inner2<Y funnel> extends Foo<Y
        >(){ Y x(){ return y; } }; } }; } }
// d11, overriding: Object, A, and
class P1<> extends Object<>{ <X extends Object<>> X id(X x){ return x; } }
class P2<> extends P1<>{ <Z extends Object<>> Z id(Z z){ return z; } }
class P3<> extends P1<>{ <Z extends Object<>> Object<> id(Z z){ return z; } }
class P4<> extends P1<>{ <Z extends A<>> Z id(Z z){ return z; } }
// d12, fields of a superclass: Object, A, B, and
class Base<X extends Object<>> extends Object<>{ X f; }
class Derived<> extends Base<A<>>{ B<> g; }
// d13, the example of Section 4.4: Object, and
class K<> extends Object<>{ Object<> k(L<> y){ return
  new L<> extends Object<>(){ Object<> m(Object<> x){ return y.m(x); } }; } }
```

In the list, the paper verdict follows from the rule or the requirement named in the item. A program that is *ill typed* has no type under the rules of Section 4, as Lean states it; a program that is *stuck* is not a value and has no reduction step.

W1 `w1`, proved: d1 satisfies A1-A11.

W2 `w2`, proved: a method with two parameters `x` violates A4.
W3 `w3`, proved: an `if` binding the parameter `x` again violates A7.
W4 `w4`, proved: an undeclared superclass violates A9 and A10.
W5 `w5`, proved: d2 satisfies A1-A11.
W6 `w6`, proved: d3 satisfies A1-A11, and the main expression of Example 1 satisfies A3-A8 and A12.
W7 `w7`, proved: d4 satisfies A1-A11.
W8 `w8`, proved: d5 satisfies A1-A11.
W9 `w9`, proved: d6 satisfies A1-A11, and the main expression of Example 2 satisfies A3-A8 and A12.
W10 `w10`, proved: d6F satisfies A1-A11.
W11 `w11`, proved: d7 satisfies A1-A11, and the main expression of Example 6 satisfies A3-A8 and A12.
W12 `w12`, proved: d8 satisfies A1-A11, and the main expression of Counterexample 7 satisfies A3-A8 and A12.
W13 `w13`, proved: d4F satisfies A1-A11.
W14 `w14`, proved: d9, with `Inner` written in a method of `Outer`, satisfies A1-A11.
W15 `w15`, proved: d10 satisfies A1-A11.
W16 `w16`, proved: the value `new Outer2<A<>> extends Foo<A<>>(){..}`, whose method `g` holds `Inner2<Y funnel>`, satisfies A3-A8 and A12: a ground funnel and a funnel in the same main expression.
W17 `w17`, proved: d11 satisfies A1-A11.
W18 `w18`, proved: a field `f` declared again in a subclass violates A2.
W19 `w19`, proved: a local class named `A`, as a top-level class, violates A1.
W20 `w20`, proved: a ground funnel in Decls violates A11.
W21 `w21`, proved: `X funnel, X funnel` violates A6.
W22 `w22`, proved: a parameter named `this` violates A8.
W23 `w23`, proved: two classes extending each other violate A9.
W24 `w24`, proved: the result of the call of `k` of Section 4.4 without renaming, a method `m(x)` holding a value whose method `m` binds `x` again, violates A7.
W25 `w25`, proved: in `if (new A<>()instanceof A<> x){ if (new B<>()instanceof B<> y){ y.onlyB()} else { new B<>()} } else { new B<>()}`, renaming `y` into `x` makes the substitution `[x=new A<>()]` of rule IFTHEN undefined, so rule IFTHEN has no instance with that renaming.
W26 `w26`, proved: `Fresh` relates `if (new A<>()instanceof A<> x){ new L<> extends Object <>(){ Object<> m(){ return x; } } } else { new B<>()}` to the same expression with `x` renamed into `this`, which the method `m` captures; the renamed expression violates A8.
W27 `w27`, proved: `Fresh` relates the program of W25 to the same program with `y` renamed into `x`; the renamed program violates A7.
W28 `w28`, proved: `if (vZ instanceof L<> x){ if (new A<>()instanceof A<> y){ x } else { x } } else { new B<>()}` violates A12, where `vZ` is `new L<> extends Object<>(){ Object<> m(){ return z; } }`.
W29 `w29`, proved: `if (vT instanceof L<> x){ new L2<> extends Object<>(){ <Y extends Object<>> Object<> k(){ return x; } } } else { new B<>()}` violates A12, where `vT` is `new L<> extends Object<>(){ Z m(){ return this; } }`.
R1 `r1`, proved: `new Box<A<>>(new A<>()).val` → `new A<>()` by SIMPLEFIELDFOUND.
R2 `r2`, proved: `new C2<>().m()` → `new C2<>().[C2<>]m()` by SIMPLEADDSUPER.
R3 `r3`, proved: `new C2<>().[C2<>]m()` → `new C2<>().[C1<>]m()` by SEARCHSUPER.

R4 `r4`, proved: `new C2<>().[C1<>]m()` → `new A<>()` by simpleFound.
R5 `r5`, proved: R1 inside the receiver of a call, by ctx.
R6 `r6Then`, proved: `if (new A<>()instanceof A<> x){ x } else { new B<>()}` → `new A<>()` by ifThen.
R6 `r6Else`, stated: the same with `B<>` as the test → `new B<>()` by ifElse.
R7 `r7`, proved: `new Maker<>().[Maker<>]<A<>>mk(new A<>())` → `new MyFoo<A<>> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }` by simpleFound, renaming `Y` and `y` into `Y1` and `y1`.
R8 `r8`, proved: the value of R7 `.x()` → `new A<>()` by innerFound.
R9 `r9`, proved: the value of R7 `.other()` → `.[Foo<A<>>]other()` by innerSuper.
R10 `r10`, proved: the value of R7 `.f` → `new Foo<A<>>().f` by innerFieldSuper.
R11 `r11`, proved: R2, R3, R4 in sequence.
R12 `r12`, stated: in d5, `new Fixed<>().<A<>>mk(new Foo<A<>>(), new A<>())` →* `new MyFoo<A<>> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }`: the type test fails by ifElse.
R13 `r13`, stated: in d5, `mk` given that value of `MyFoo<A<>>` returns it: the type test succeeds by ifThen.
R14 `r14`, stated: Example 2 reduces to `new A<>().[Object<>]onlyB()`, which is stuck: the second call finds the `MyFoo2` of the first call, and `AsB` receives the `A<>` of the first call.
R15 `r15`, stated: Example 6 reduces to `new A<>().[Object<>]onlyB()`, which is stuck: `mine.show(local)` applies the `f` of the second call to the `v()` of the first call.
R16 `r16`, stated: Counterexample 7 reduces as Example 2, to `new A<>().[Object<>]onlyB()`, which is stuck.
R17 `r17`, stated: `new Leaker<>().<B<>>leak(new Leaker<>().<A<>>mk2(new A<>())).x().onlyB()` reduces to `new A<>().[Object<>]onlyB()`, which is stuck.
R18 `r18`, stated: `new Nest<>().<A<>>mk(new A<>()).x()` →* `new A<>()`, through a value of `Outer` and a value of `Inner`.
R19 `r19`, stated: calling `g<B<>>` on the value of W16 gives `new Inner2<B<>> extends Foo<B<>>(){ B<> x(){ return new B<>(); } }`.
R20 `r20`, proved: `new Derived<>(new B<>(), new A<>()).f` → `new Base<A<>>(new A<>()).f` → `new A<>()`, by simpleFieldSuper and simpleFieldFound.
R21 `r21`, proved: `if (new A<>()instanceof A<> x){ if (x instanceof A<> y){ y } else { x } } else { new B<>()}` → `if (new A<>()instanceof A<> y1){ y1 } else { new A<>() }`: ifThen renames the inner `y`.
R22 `r22`, proved: `new K<>().k(vL)` →* `new L<> extends Object<>(){ Object<> m(Object<> x1){ return vL.m(x1); } }`, where `vL` is a value of `L` whose method `m` binds `x`: simpleFound renames `y` and `x`, so the result satisfies A7 (compare W24).
R23 `r23`, proved: the program of W25 reduces to `new B<>()` in four steps, renaming `y` into `y1`.
T1 `t1`, proved: `new Box<A<>>(new A<>()).val : A<>`.
T2 `t2`, stated: `new Box<A<>>(new B<>())` is ill typed: `B<>` is not a subtype of `A<>`.
T3 `t3`, stated: in d5, `new Fixed<>().<A<>>mk(new Foo<A<>>(), new A<>()): Foo<A<>>`.
T4 `t4`, proved: `new Derived<>(new B<>(), new A<>()).f : A<>`, by subs to `Base<A<>>` and then declField.
C1 `c1`, proved: `X extends Object<>` ⊢ `Foo<X>` ⇝ `MyFoo<X>` in d2, by cast-down.
C2 `c2`, stated: not `X extends Object<>` ⊢ `Object<>` ⇝ `MyFoo<X>` in d2: cast-down needs $\texttt{Y} \in \mathsf{FTV}(\emptyset)$, since generic(`MyFoo`) is `MyFoo<Y>`, and `MyFoo<Y>` has the supertype `Object<>` with no type arguments.

C3 `c3`, proved: `X extends Foo<A<»` $\vdash$ `X` $\rightsquigarrow$ `MyFoo<A<»` in d2, by CAST-VAR and then CAST-DOWN.
C4 `c4`, proved: $\emptyset \vdash$ `B<>` $\rightsquigarrow$ `Object<>`, by CAST-UP.
C5 `c5`, stated: not $\emptyset \vdash$ `A<>` $\rightsquigarrow$ `B<>`: the classes are unrelated.
D1 `d1Inner`, proved: `Inner` is declared: it occurs in the method `x` of `Outer`, which occurs in the method `mk` of `Nest`.
K1 `k1`, proved: $\emptyset \vdash$ `Inner<A<»` Ok: the scope of `Inner` goes through the scope of `Outer<X>`.
O1 `o1`, proved: overrideOk(`P1<>`, M) for the method `id` of `P2`, whose type variable is `Z` instead of `X`.
O2 `o2`, stated: not overrideOk(`P1<>`, M) for the method `id` of `P3`, which returns `Object<>`.
O3 `o3`, stated: class `P2` is Ok.
O4 `o4`, stated: classes `P3` and `P4` are not Ok: `P3` changes the return type, `P4` changes the bound of the type variable.
M1 `m1`, proved: class `Maker` is Ok.
V1 `v1`, proved: the hypotheses of `typeSoundness` hold together on d1 and the expression of T1; in particular every class of d1 is Ok.
V2 `v2`, proved: the hypotheses of `typeSoundness` hold together on d2, with the funnelled local class `MyFoo`, and `new Maker<>().<A<>>mk(new A<>()): Foo<A<>>`.
V3 `v3`, proved: the result of R7, a copy of `MyFoo` in the main expression, has type `Foo<A<>>`.
P1 `p1`, stated: class `Fixed` is Ok.
P2 `p2`, stated: class `Fixed2` is Ok: `Holder<X>` determines `X`.
P3 `p3`, stated: class `LeakerF` is Ok.
N1 `n1`, stated: Example 1 is ill typed: rule NEW-LOCAL fails at $\emptyset \vdash$ `Foo<X>` Ok, since `MyFoo` funnels nothing.
N2 `n2`, stated: in d5, the main expression of Example 1 is ill typed: a `Foo<A<>>` is passed where a `Foo<B<>>` is expected.
N3 `n3`, stated: Example 2 is ill typed: the method `x` of `MyFoo2` returns `X`, which `MyFoo2` does not funnel.
N4 `n4`, stated: class `Fixed2Obj` is not Ok: `MyFoo4` funnels `X`, but `Object<>` does not determine `X`, so the type test is rejected.
N5 `n5`, stated: Example 6 is ill typed: `MyFoo6` mentions `X` and captures `f`, but funnels nothing.
N6 `n6`, stated: class `Box7` is not Ok: `MyFoo7` does not funnel the class type variable `X`.
N7 `n7`, stated: class `Box7F` is not Ok: `Object<>` does not determine `X`; javac rejects Counter-example 7 for the same reason.
N8 `n8`, stated: in d4F, the main expression of the leak is ill typed: a `MyFoo2F<A<>>` is passed where a `MyFoo2F<B<>>` is expected.
J2 `j2`, stated: Example 1 reduces to `new A<>().onlyB()`, and then to `new A<>().[Object<>] onlyB()`, which is stuck.
L2 `l2`, stated: class `Leaker` is not Ok: the local class in `mk2` funnels nothing, as in N1.
S1 `s1`, stated: `new Object<>().f` is stuck.

References

1 Nada Amin and Ross Tate. Java and scala's type systems are unsound: The existential crisis of null pointers. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, page 838–848, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2983990.2984004.

2 Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005.

3 Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and gj. *ACM Trans. Program. Lang. Syst.*, 23(3):396–450, 5 2001. doi:10.1145/503502.503505.

4 Marco Servetto, Nick Webster, and Colin S. Gordon. Java method-local inner classes are unsound. Talk at UNSOUND 2024, co-located with SPLASH 2024, October 2024. Recording at https://www.youtube.com/watch?v=DfkOpk733dQ. URL: https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound.